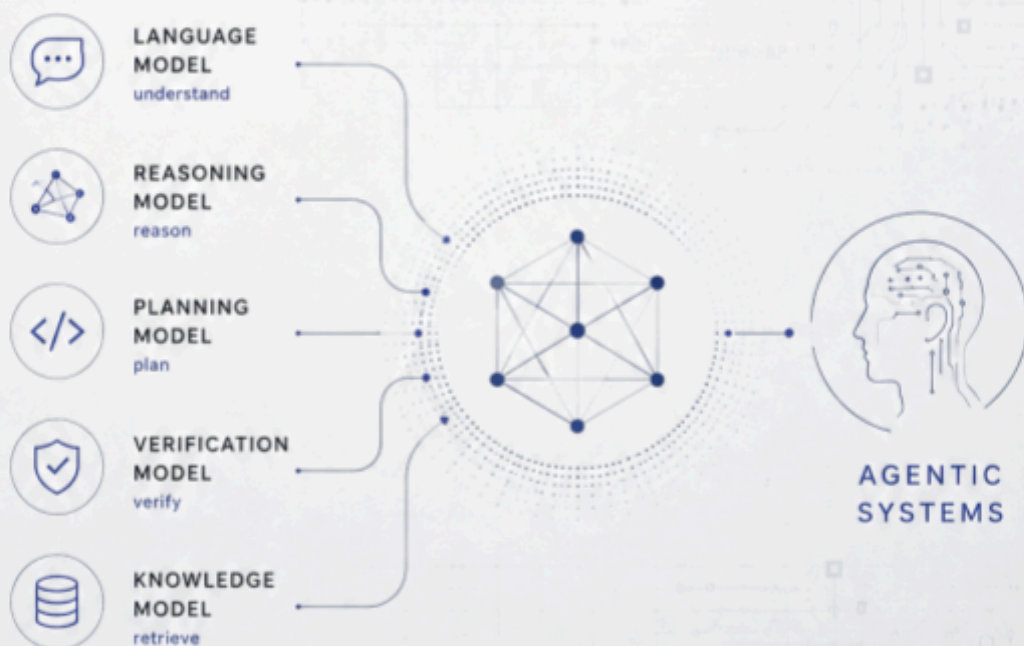


S*LM

INTELLIGENCE BY DIVISION OF LABOUR



SMALL, SPECIALIZED, AND SYMBOLIC LANGUAGE MODELS
FOR AGENTIC SYSTEMS



INTELLIGENCE BY DIVISION OF LABOUR

Small, Specialized, and Symbolic Language Models for Agentic Systems

A Survey, Architectural Synthesis, and Research Programme

Axiologic Research Programme

Evidence reviewed to 25 August 2026

Copyright © [2026] Axiologic Research

All rights reserved.

KDP publishing rights held by Outfinity SRL (Iași, Romania).

Available for free on Axiologic website (www.axiologic.net).

Author's Note

This work was created under the umbrella of Axiologic Research as part of two interconnected research programs, one dedicated to Artificial Intelligence and the other to Outfinitism, both led by Sînică Alboaie. To explore the details of how these two programs intersect and build upon each other, we invite readers to visit the Outfinity Venture Studio page at www.outfinity.ch.

While Artificial Intelligence language models were used to assist with text generation, the foundational philosophy, thematic structure, and analytical approach derive exclusively from the author's decades of dedicated study of social phenomena, social technologies, and genuine hard technologies resulting from various European and self funded research projects. Recently, within the Achilles research project, we have been deeply studying AI generated literature and its automated verification using neuro symbolic approaches; all the free books made available to the public are part of the suite of works we use to test our latest technologies.

Abstract

The dominant story of language-model progress has been a story of scale. Agentic systems require a different story: one about the division of cognitive labour. A useful agent does not merely produce prose. It identifies what a person wants, interprets entities and constraints, selects tools, constructs formal requests, retrieves and updates state, checks outputs, manages authority, and explains what happened. These operations have unequal requirements. Some need broad linguistic and world knowledge. Others are repetitive classification problems. Some require exact computation rather than plausible text. Some must run locally because of latency, privacy, availability, or cost. Treating all of them as one prompt to one frontier model is technically convenient, but it obscures the structure of the problem.

This book names a research programme around **S*LM systems**. The star is a wildcard: a component may be small, specialized, symbolic, or combine these properties. The term does not pretend that an SMT solver or planner is a language model. It denotes a system family in which linguistic intelligence is decomposed across compact neural models, task-specific models, constrained representations, symbolic engines, deterministic software, and selective access to more general models. The research question is not whether small models can replace large models in the abstract. It is how much generality each role actually needs, how roles should be composed, where uncertainty must trigger escalation, and how authority can remain outside probabilistic text generation.

The survey finds strong evidence that compact models are already useful for bounded language operations and increasingly viable for function calling, routing, extraction, embeddings, moderation, on-device interaction, and domain adaptation. FunctionGemma, compact Phi, Qwen, Gemma, Llama, Granite, SmolLM, Ministral, DeepSeek-distilled and Apple on-device families illustrate a broad industrial move toward smaller deployable models, while xLAM, Hammer, APIGen and ToolACE show that

verified or carefully synthesized tool-use data can produce disproportionate gains in narrow action policies. The academic evidence for neural-symbolic composition is also substantial at the architectural level: MRKL, Logic-LM, LINC, SatLM, LLM+P and LLM-Modulo all separate linguistic interpretation from exact or checkable computation. Yet the evidence does not justify a triumphalist conclusion. Formalization remains fallible; compact models can be brittle outside their training envelope; routing often fails to recognize when the small model is sufficient; and multi-agent systems frequently add coordination errors rather than independent intelligence.

The central judgement is therefore conditional. S*LM is already a credible engineering pattern for repetitive, verifiable and authority-bounded agent work. It becomes a deeper research programme when it asks whether recurring language-mediated work can be progressively compiled into reviewed models, schemas, circuits, memories and validators while preserving semantic fidelity. If that direction succeeds, the important product will not be one more small chatbot. It will be an agent runtime that allocates interpretation, reasoning, memory, execution and authority to the least general component that can perform each operation reliably.

Method and Evidence Discipline

This is a survey and a position, not a model catalogue disguised as a theory. The literature review prioritizes original papers, model cards, official technical reports, repositories, and operational guidance from model and platform developers. The evidence is current to 25 August 2026. Vendor benchmark claims are treated as evidence that an organization is investing in a direction, not as independent proof that the claimed system dominates alternatives. Preprints are used when they expose designs, data or failure analyses that matter to the programme, but they are identified as provisional where appropriate.

Four labels govern the argument. **Established practice** refers to capabilities repeatedly demonstrated in deployed software or across independent studies, such as compact embedding models, constrained JSON generation, external execution of function calls, and quantized local inference. **Convergent signal** refers to a pattern supported by several research groups or vendors but still sensitive to task design, such as the use of small action models within larger agent stacks. **Research hypothesis** marks a proposition that is plausible but not yet established, such as the progressive compilation of recurring agent trajectories into a library of compact policies and symbolic procedures. **Conditional outcome** states what could become possible if the hypothesis survives demanding evaluation; it is not presented as a forecast.

The book deliberately avoids benchmark score tables. Scores are often useful inside a controlled engineering decision, but weak as the organizing principle of a research programme. They vary with prompts, harnesses, tool schemas, quantization, sampling, hardware and data contamination. The analysis instead asks what a result reveals about the allocation of work. A one-billion-parameter model outperforming a larger model on a function-calling suite is not evidence that it is generally smarter. It is evidence that task-specific data, output contracts and narrow action spaces can matter more than parameter count for that role. Conversely, a large model leading a broad benchmark does not prove that every high-volume substep of an agent should use it.

The scope is language-centred but not text-only. Speech, images and graphical interfaces are included where they create or consume linguistic intent. Symbolic systems include logic, constraints, planning, databases, knowledge graphs, static analysis, program execution and formal verification. They are included because they change what the neural model must learn and what the system can guarantee. The decisive distinction is between generating a candidate and granting that candidate authority. Neural models may propose interpretations and actions. Deterministic components, policies and humans decide whether those proposals are admissible and whether their side effects may be committed.

Evidence label	Meaning in this book
Established practice	Repeatedly demonstrated in deployed systems or across independent technical work; still conditional on the named role and environment.
Convergent signal	Several academic or industrial sources point in the same direction, but the result remains task-sensitive or lacks standardized evaluation.
Research hypothesis	A plausible, testable proposition that is not presented as established.
Conditional outcome	What could become possible if the hypothesis survives demanding evaluation; not a forecast or product promise.

Contents

Section	Title
PART I	The S*LM Position
Chapter 1	From Scale to Division of Cognitive Labour
Chapter 2	The S*LM Taxonomy: Size, Specialization, and Symbolic Structure
Chapter 3	Contracts, Authority, and the Heterogeneous Agent Runtime
PART II	The Agentic Language Stack
Chapter 4	Reading the Request: Intent, Routing, Ambiguity, and Multilingual Input
Chapter 5	Language Transformations as Agent Infrastructure
Chapter 6	From Natural Language to Formal Objects
Chapter 7	Planning, Tool Use, and Controlled Action
Chapter 8	Memory, Retrieval, Verification, Recovery, and Escalation
Chapter 9	Multi-Agent Systems Without Role-Play
PART III	Trust, Deployment, and Economics
Chapter 10	Trustworthy AI by Architecture
Chapter 11	Training, Deployment, and the Economics of Successful Work
PART IV	Survey Synthesis and Research Programme
Chapter 12	What Academia and Industry Are Actually Signalling
Chapter 13	The S*LM Research Programme
Appendix A	A Role Taxonomy for S*LM Components
Appendix B	Representative Model and Specialist Families
Appendix C	Symbolic, Formal, and Deterministic Backends
Appendix D	Deployment and Agentic Infrastructure
Appendix E	Research Hypotheses and Falsification Conditions
References	Selected literature, model documentation, standards and technical systems

PART I

The S*LM Position

The programme begins by replacing the question “How small can a general model be?” with the more useful question “How much generality does each operation require?”

CHAPTER 1

From Scale to Division of Cognitive Labour

The modern language-model industry was built around a persuasive empirical fact: increasing scale, data quality and training compute repeatedly produced more capable general models. That fact remains important. It does not, however, determine the best architecture for every agentic system. An agent is not a benchmark answerer with access to buttons. It is a controlled process that moves from an underspecified human request to state changes in software, databases, instruments or organizations. Once side effects, cost, privacy and accountability matter, the relevant unit of analysis is no longer the model in isolation. It is the allocation of work across the complete system.

Consider an ordinary enterprise request: “Find the suppliers affected by the new restriction, estimate our exposure, and prepare the changes, but do not send anything.” A monolithic implementation sends the request, policy documents, supplier records and tool descriptions to one powerful model, then asks it to reason and act. A decomposed implementation treats the sentence as several different problems. It detects the requested outcome and the prohibition on sending. It resolves which restriction is meant. It retrieves the authoritative policy version. It maps supplier names to internal identifiers. It executes tested queries. It calculates exposure with ordinary code. It drafts proposed changes. It validates that no communication tool can be called. It preserves the evidence used for every conclusion. A large model may still be valuable where the language is genuinely ambiguous, but most of the workflow is not improved by asking the same model to rediscover database semantics, arithmetic and permissions in every run.

This distinction explains the S*LM notation. “Small” describes a resource envelope, not an epistemic virtue. “Specialized” describes a

deliberately narrow distribution of tasks or outputs. “Symbolic” describes explicit representations and procedures whose behavior is governed by rules, constraints or executable semantics. The star permits combinations. A compact model fine-tuned to translate natural-language requests into a typed function schema is both small and specialized. A domain model that emits a controlled language checked by a solver is specialized and symbolically coupled. A planner or theorem prover is symbolic but is not itself a language model. The programme studies the composition, not the branding.

Established practice. Small models already occupy valuable roles that are narrow, frequent and easy to validate. Embedding, reranking, classification, moderation, entity extraction, language identification and short-form transformation have long been served by task-specific neural models. The new development is that compact generative models can now absorb more flexible instructions, produce structured outputs and call functions while remaining deployable on consumer or edge hardware. The Phi, Gemma, Qwen, Llama, SmoLLM, Granite, Ministral and Apple model lines show sustained industrial investment in that capability, while MobileLLM and related research examine architectures designed specifically for sub-billion and on-device regimes. [1, 2, 3, 7, 9, 11, 15, 18, 19, 20]

The practical significance of this trend is not that a three-billion-parameter model has become a miniature frontier model. It is that more agent roles have crossed the threshold from “requires a remote general model” to “can be handled by a local or inexpensive component under a clear contract.” Google’s FunctionGemma makes this logic unusually explicit. It is a 270-million-parameter model intended as a foundation for function-calling applications and further task adaptation, not as a broad conversational assistant. The design says something more important than any headline score: a useful action policy may need a small language interface and a narrow output grammar rather than encyclopedic conversational competence. [13, 14]

Academic position. Research on modular and neuro-symbolic systems has argued for division of labour since before the present wave of compact generative models. MRKL proposed combining language models with

external modules and discrete reasoning. ReAct made interaction visible by alternating reasoning and action. Toolformer, ToolLLM and Gorilla explored how models can learn when and how to invoke external capabilities. Logic-LM, LINC, SatLM, LLM+P and LLM-Modulo separated linguistic interpretation or candidate generation from solvers, planners and verifiers. These works differ substantially, but they converge on one architectural lesson: language models are often most valuable at the boundary between informal intent and formal computation, not as substitutes for every computational substrate. [26, 25, 24, 27, 28, 35, 36, 37, 38, 39]

That convergence should not be overstated. A solver can prove that a formal encoding is satisfiable while the encoding misrepresents the user's request. A planner can return a valid action sequence for an incomplete world model. A grammar can guarantee JSON syntax while allowing a dangerous or nonsensical tool call. External computation reduces the amount of exact reasoning a neural model must approximate, but it moves epistemic risk toward translation, state construction and interface design. The difficult question is therefore not merely whether symbolic methods improve reliability. It is where they create a genuine guarantee and where they only create the appearance of one.

Industry signal. Model providers increasingly release families rather than single general checkpoints. Google's Gemma ecosystem includes compact general models, function-calling, embedding, safety, translation and medical variants. IBM presents Granite as a collection spanning language, speech, vision, embedding and guardian functions, and explicitly frames predictable latency and tool use as enterprise priorities. Mistral releases small dense models in base, instruct and reasoning variants for local and edge deployment. Apple trains models for distinct on-device and private-cloud roles and optimizes each for its hardware target. IBM's later "AI more like software" framing goes further by treating adapters and capabilities as composable modules. These are vendor positions, not neutral scientific conclusions, but together they show that industry is moving from one-model product narratives toward portfolios of role-specific components. [15, 20, 19, 18]

The strongest reason for this movement is not parameter efficiency alone. Production systems contain repeated distributions. A customer-service agent sees recurring intents, policies and databases. A scientific assistant repeatedly extracts claims, methods, datasets and uncertainty. A development agent repeatedly selects files, edits code, runs tests and interprets failures. Repetition creates the possibility of specialization: examples can be collected, errors can be categorized, outputs can be typed, and recurrent decisions can be compiled into models or code. Open-ended novelty weakens that advantage because the system has less evidence about what the next task will require. The architecture should therefore specialize around stable distributions and escalate around novelty.

Research judgement. The relevant optimization target is minimum sufficient generality. A component should possess enough breadth to handle the variation in its assigned role, but not so much unbounded authority that every request becomes an exercise in unconstrained generation. This differs from choosing the smallest available model. A weak model that misroutes requests or produces repeated invalid calls may cost more than a larger model that succeeds once. It also differs from choosing the strongest model by default. A frontier model applied to millions of repetitive classifications may add cost, latency, privacy exposure and behavioral variability without adding useful competence.

The notion of minimum sufficient generality leads to a concrete design discipline. Every agent role should have a defined input distribution, output contract, failure envelope, authority level and escalation path. The model is selected after those are understood. If the output can be expressed as a closed set of intents, a classifier may be enough. If it requires flexible but schema-bounded arguments, a compact generative action model may be appropriate. If it requires exact search under explicit constraints, a symbolic engine should own the search. If the task demands broad synthesis across unfamiliar concepts, a general model may be justified. If consequences are irreversible, no model's fluency should itself grant execution authority.

Research hypothesis. Recurrent agent work can be progressively migrated from general inference into specialized and checkable

capabilities. Early deployments may depend heavily on a general model to interpret requests, propose plans and generate examples. As trajectories accumulate, stable subproblems can be isolated, labelled, tested and served by smaller models, adapters, deterministic functions or symbolic procedures. The result is not a fixed workflow that eliminates intelligence. It is a system that learns which parts of its workload have become routine enough to deserve compilation.

What present results signal. Function-calling datasets such as APIGen, xLAM, Hammer and ToolACE indicate that narrow action behavior can improve sharply when training data reflects executable tool contracts rather than generic instruction following. Routing studies show that model complementarity is real, but also that many learned routers remain close to simple baselines and far from an oracle that always knows which model will succeed. Multi-agent failure analyses show that adding agents introduces new coordination and verification problems. Taken together, these results support decomposition but reject naïve modularity. The modules, contracts and router must be engineered and evaluated as seriously as the models. [29, 30, 31, 32, 47, 48, 52]

Conditional outcome. If the research programme succeeds, an enterprise or scientific agent will not be defined by one model name. It will be defined by a governed capability graph. Compact models will perform high-volume interpretation and transformation. Specialized models will handle domain language and action grammars. Symbolic and conventional software will own exact computation, state transitions and policy enforcement. Larger models will be invoked when uncertainty, novelty or synthesis requirements justify them. Humans will retain authority over consequential commitments. The gain would not be a mystical form of distributed cognition. It would be a more legible, economical and falsifiable way to build language-mediated software.

CHAPTER 2

The S*LM Taxonomy: Size, Specialization, and Symbolic Structure

A useful taxonomy must prevent three recurring confusions. The first is to equate smallness with a fixed parameter threshold. The second is to call every domain-tuned model specialized in the same sense. The third is to describe any agent that invokes a solver as a symbolic language model. These shortcuts obscure the design choices that actually determine behavior.

Model size is best treated as a deployment-relative quantity. Parameter count matters because it affects memory, bandwidth and compute, but active parameters, context length, numerical precision, batch size, architecture and hardware often matter as much. A sparse model may store many parameters while activating relatively few per token. A dense three-billion-parameter model with a very long context can consume more working memory than its weight file suggests. A 270-million-parameter function caller can be operationally sufficient for a narrow mobile action space, while a model of similar size may be unusable for multilingual legal interpretation. “Small” therefore means small enough to change the deployment regime: local execution, lower-cost high-volume serving, reduced energy, faster startup, or placement near the data source.

The current ecosystem contains at least four small-model design strategies. Some models are trained from scratch for compact regimes, as in TinyLlama, SmolLM and MobileLLM. Some are distilled or pruned from larger teachers, as in compact Llama variants and the DeepSeek-R1 distilled family. Some use mixture-of-experts or hybrid architectures to increase stored capacity without activating the entire network for each token, as seen in parts of the Granite and Nemotron ecosystems. Some are

trained as components within a product family, where smallness is paired with explicit roles such as embedding, safety, translation or function calling. These strategies are not interchangeable. From-scratch models offer cleaner experimental control; distillation imports capabilities and biases from a teacher; sparse models alter memory and serving trade-offs; family models benefit from shared tooling and aligned interfaces. [4, 5, 6, 3, 11, 23, 20, 21, 15]

Specialization also has several meanings. **Task specialization** narrows the operation: classification, extraction, translation, reranking, summarization, code completion or function calling. **Domain specialization** narrows the language and knowledge distribution: medicine, law, finance, cybersecurity, chemistry or a specific company's product catalogue. **Interface specialization** narrows the output language: a schema, controlled language, query dialect, tool API or command grammar. **Policy specialization** narrows acceptable behavior: a safety classifier, compliance checker or privilege gate. **Hardware specialization** adapts architecture and quantization to a target such as a phone NPU, embedded GPU or browser runtime. A model may be specialized along several axes, and each axis creates a different maintenance burden.

Task specialization usually yields the clearest validation. A function caller can be checked for tool choice, argument validity and abstention. A translation model can be evaluated for meaning preservation in defined language pairs and domains. A medical model is more difficult because “medical” covers extraction, question answering, image understanding, diagnostic support and patient communication, each with different risks. Domain labels are therefore weaker than role contracts. MedGemma, BioMistral, LEGAL-BERT and FinGPT show sustained interest in domain models, but their value depends on the precise task, evidence source, user population and governance context. A domain model is not a licensed professional, and domain pretraining does not confer authority. [72, 71, 70, 69]

Symbolic structure begins where the system exposes representations and operations that can be inspected independently of next-token probabilities. A JSON Schema constrains fields and types. SQL describes a database operation. PDDL defines objects, predicates, actions and goals for

a planner. An SMT formula states constraints to a solver. A knowledge graph makes entities and relations explicit. A controlled natural language restricts syntax and semantics to improve mapping into logic or code. Program execution, static analysis and tests provide additional symbolic or deterministic checks. None of these systems understands the user's language by itself. Their value is that, once the mapping is accepted, they can apply explicit procedures rather than imitate the result in prose.

The term “symbolic language model” should therefore be used cautiously. There are at least three defensible objects behind it. The first is a language model trained to emit symbolic forms. The second is a hybrid system in which a language model supplies perceptions or translations to a symbolic reasoner. The third is a persistent symbolic capability library that behaves like a model of a domain because it stores reusable concepts, rules, procedures and derivations. The last object can be scientifically interesting, but it should not be confused with a neural model or claimed to possess language understanding merely because a language model writes into it.

Established practice. Constrained generation engines such as XGrammar and Outlines can enforce syntactic structures during decoding. Databases, planners and solvers can execute or check the resulting artifact. Local runtimes such as llama.cpp, ONNX Runtime, ExecuTorch, LiteRT and MLX make compact models deployable across CPUs, GPUs and device accelerators. This creates a practical stack in which a small model produces a typed candidate and a deterministic component decides whether the candidate is well formed. [42, 44, 76, 77, 78, 79, 80]

Important limit. Syntactic validity is not semantic fidelity. A grammar may guarantee that a purchase request contains an amount and supplier identifier while failing to establish that the supplier is the one the user intended. A SQL parser may accept a query that joins the wrong tables. A theorem prover may verify a proposition that is a weaker surrogate for the source statement. Research in semantic parsing and autoformalization repeatedly exposes this boundary: compilation success proves a property of the formal artifact, not that the artifact faithfully represents the original language. [86, 35, 36]

The most useful taxonomy is consequently role-based. At the **perception boundary**, models identify language, transcribe speech, read images or ground interface elements. At the **interpretation boundary**, they detect intent, entities, constraints and ambiguity. At the **transformation layer**, they translate, normalize, summarize, extract, redact or rewrite while preserving specified invariants. At the **formalization boundary**, they emit schemas, queries, plans, programs or logical theories. At the **decision layer**, planners, routers and policies select the next operation. At the **execution layer**, tools and deterministic software change state. At the **evaluation layer**, critics, tests, solvers and human reviewers assess results. At the **memory layer**, databases preserve facts, trajectories, permissions and provenance. A single general model can occupy several roles, but the taxonomy makes visible when that choice is unnecessary or risky.

This role view also clarifies the relationship between compact general models and specialized models. A compact general model is valuable where the role must tolerate diverse language but modest task depth. It can be a router across a known tool collection, a local conversational interface, or a first-pass extractor that escalates uncertain cases. A specialized model is preferable where the output space is stable and high volume justifies training and maintenance. A symbolic component is preferable where exactness, search, state consistency or policy enforcement is central. A large general model remains preferable when the role requires broad synthesis, unfamiliar concepts, cross-domain analogy or recovery from novel situations.

Research judgement. Specialization should be justified by entropy reduction. The more the role reduces uncertainty about inputs, outputs, permissible actions and evaluation, the more likely a small or symbolic component can perform it reliably. A closed set of ten intents has low output entropy. Mapping arbitrary scientific prose into a faithful formal theory has high semantic entropy even if the target syntax is constrained. This explains why tiny function callers can be credible while tiny autonomous scientists remain speculative. The problem is not simply task difficulty; it is how much variation and hidden context the component must absorb.

There is also a lifecycle taxonomy. A component can be **static**, with fixed weights and rules; **adapted**, through LoRA, QLoRA or another parameter-efficient method; **retrieval-conditioned**, with external knowledge selected at runtime; **continually curated**, with updated memory and rules; or **dynamically routed**, with work allocated among multiple models. Each form has different failure modes. Static models become stale. Adapters can interfere or drift. Retrieval introduces source and injection risks. Curated symbolic knowledge creates governance and versioning work. Dynamic routing can choose the wrong expert. The programme should compare these alternatives at the system level rather than assume that one learning mechanism is always superior.

What academia is investigating. Surveys now map compact models across architecture, data, training, compression and deployment. Tool-use papers examine how data generation and verifiable execution can create action specialists. Neural-symbolic research studies translation into logic and planning. Routing research studies the complementarity of model pools. Memory research is moving from prompt history toward lifecycle and data-management questions. Reliability research is beginning to separate average task success from consistency, robustness, predictability and severity of failure. These lines are usually published separately, yet they describe adjacent parts of the same agent runtime. [1, 29, 35, 38, 48, 53, 54, 61]

What industry is building. Vendors are releasing compact general models, domain variants, embedding models, guard models, on-device frameworks, quantized checkpoints and tool-calling interfaces. The commercial pattern is less a return to classical symbolic AI than a pragmatic stratification of the stack. Natural language remains neural and flexible. Execution remains software. Safety and policy are increasingly separated into guard components. Hardware constraints shape model architecture. The vocabulary varies, but the design pressure is consistent: not every token and action should pass through the most expensive general model.

Research hypothesis. The S*LM taxonomy can become an engineering decision procedure rather than a descriptive catalogue. Given a role, one should be able to characterize its semantic breadth, output entropy, verification strength, authority, latency budget, privacy class and rate of

repetition. Those properties should predict whether the role belongs in a classifier, compact generator, domain adapter, symbolic engine, general model or human review process. The hypothesis is testable: architectures selected by this procedure should achieve lower cost and equal or better governed task success than naïve one-model baselines across multiple domains.

Conditional outcome. If the taxonomy proves predictive, model choice will become less fashion-driven. Teams will stop asking whether a named small model is “good enough” in the abstract and will ask whether its validation envelope covers a specific role. Symbolic systems will be valued for the guarantees they genuinely provide rather than for rhetorical claims of reasoning. General models will remain central, but their use will be concentrated where generality has measurable value.

CHAPTER 3

Contracts, Authority, and the Heterogeneous Agent Runtime

The architectural unit of an S*LM system is not the agent persona. It is the contract. A contract states what a component may read, what it may produce, how its output is checked, what resources it may consume and what authority follows from success. This may sound like ordinary software engineering, but it corrects a persistent weakness in agent design: prompts often mix intent, policy, data, implementation and authorization into one natural-language context, then ask a probabilistic model to respect all of them simultaneously.

Return to the supplier example. The intent interpreter may read the user request and a catalogue of supported operations. Its output is a typed object containing the requested analysis, the relevant policy reference and an explicit “draft only” constraint. The retrieval component may read that object and a document index, but not communication credentials. The database procedure may accept validated supplier identifiers and return records, but it cannot invent identifiers. The exposure calculator may accept structured records and a versioned formula. The drafting model may read the results and approved templates, but it cannot send messages. The final action gate can verify that every proposed side effect is disallowed under the request. No single prompt has to serve as both instruction and security boundary.

Established practice. The separation between model output and tool execution is already standard in mature function-calling APIs. The model proposes a function and arguments; application code validates and executes them. Structured generation frameworks improve syntactic conformance, while workflow engines and agent SDKs preserve state, checkpoints and human approval. Least-privilege security research such as Progent goes further by expressing tool-call permissions as enforceable

policies rather than polite instructions. NIST and OWASP guidance similarly treat governance, identity, access, monitoring and incident response as system properties. [42, 43, 60, 57, 58]

The contract view divides an agent runtime into six responsibilities. The first is **interpretation**, where linguistic input becomes candidate intent and constraints. The second is **context construction**, where the runtime selects the evidence, memory and tool descriptions required for the next operation. The third is **capability selection**, where a router chooses a model, function, solver or human. The fourth is **execution**, where a bounded component performs the operation. The fifth is **evaluation**, where outputs are checked against schemas, tests, policies, evidence and task state. The sixth is **commitment**, where an authorized actor changes persistent state or communicates externally. These responsibilities may be implemented in many ways, but collapsing them removes useful control points.

A central design principle follows: proposal and authority must be separate. A model can propose a database query, but database credentials and row-level access remain outside the model. It can propose a code patch, but a sandbox, tests and review decide whether the patch enters a repository. It can propose a clinical extraction, but source verification and a qualified user determine whether the interpretation affects care. It can propose a plan, but a policy engine decides which actions are allowed and a human may approve irreversible steps. The stronger the consequence, the less reasonable it is to treat fluent generation as authorization.

This separation also changes how errors are handled. In a monolithic system, a wrong answer may be difficult to localize because interpretation, retrieval, reasoning and composition occurred inside one generation. In a contract system, failures can be assigned to boundaries. The intent object omitted a negation. The retriever selected an obsolete policy. The formalizer used the wrong unit. The planner assumed an action was reversible. The verifier checked syntax but not semantics. This localization does not automatically prevent failure, but it makes improvement cumulative. A corrected contract, test or capability can benefit every future task that uses the same boundary.

Two timescales. S*LM systems should distinguish slow acquisition from fast execution. During acquisition, powerful models and coding agents may read broad corpora, propose schemas, generate training examples, synthesize adapters, discover rules and construct tests. Outputs remain candidates until reviewed. During execution, the runtime should prefer promoted capabilities with known interfaces and validation envelopes. It may instantiate a reviewed template or call an approved model, but it should not silently install newly generated code or policy into a consequential workflow. This resembles compilation, though the source material is underspecified language rather than a complete program.

The two-timescale architecture provides a realistic path to specialization. A new domain begins with extensive use of general models because little is formalized. Recurrent trajectories reveal stable operations. Engineers collect representative and adversarial examples, define outputs, train compact models or adapters, implement deterministic procedures, and add validators. Once promoted, the capability handles routine cases. New or uncertain cases still escalate. Specialization is therefore not a one-time choice between “small” and “large.” It is the result of an evidence-producing lifecycle.

Academic signal. APIGen is important less because of a particular leaderboard result than because it treats executable verification as a data-generation filter. xLAM, Hammer and ToolACE similarly focus on action distributions and robustness rather than generic prose. Logic-LM and related systems use solver feedback to repair formalizations. LLM-Modulo makes the verifier a first-class component of generation. These works suggest that the quality of interfaces and feedback may be as important as raw model scale for bounded agent roles. [29, 30, 31, 32, 35, 39]

Industry signal. Apple exposes guided generation, tool calling and adapters around on-device models. Google positions FunctionGemma for application-specific function calling and provides edge deployment paths. IBM emphasizes modular model families and adapter composition. Mistral distributes compact variants in formats intended for local and edge environments. The common product insight is that models become more

useful when embedded in a software contract: a set of tools, schemas, adapters, hardware targets and privacy assumptions. [13, 14, 17, 18, 20, 19]

A contract must include abstention. Compact models often fail not because they lack a narrow skill, but because they apply it beyond its validation envelope. An intent classifier trained on twenty enterprise operations should not improvise a twenty-first. A medical extractor should identify unsupported document types. A formalizer should be able to return “ambiguous mapping” rather than choose a plausible predicate. Abstention is not a conversational apology. It is a typed outcome that triggers clarification, another component or human review.

A contract must also include uncertainty that is meaningful at the system level. Token probability alone is a weak proxy for operational confidence. Better signals may include disagreement among independently trained components, failure to retrieve a matching tool, violation of a schema, solver inconsistency, distance from known examples, historical error rates for the current intent, or inability to satisfy policy constraints. A router can combine these signals to decide whether to proceed locally or escalate. Routing research shows that this decision remains difficult: model pools are complementary, but current routers often miss which model will succeed. [46, 47, 48]

Memory belongs inside the contract architecture, not beside it. A component should not receive an unbounded transcript merely because it participated earlier in the task. It should receive task-relevant state with provenance, version and sensitivity. Writes should have schemas and authority. Derived beliefs should remain distinguishable from user statements and verified facts. Checkpoints should permit rollback after a failed action. The growing literature on agent memory increasingly treats storage, extraction, retrieval, routing and maintenance as distinct system problems, which is closer to database engineering than to conversational history. [53, 54]

Research judgement. The contract is where the three S dimensions become operational. Smallness becomes a resource and deployment clause. Specialization becomes a declared validation envelope. Symbolic structure becomes an explicit representation, checker or executor.

Without contracts, a collection of small models is merely a fragmented prompt chain. With contracts, heterogeneity can reduce risk because each component owns a narrower obligation and less authority.

The contract architecture also defines when multi-agent language is useful. If two components use different tools, models, memories, objectives or authorities, describing them as agents may clarify the system. If they are identical prompts given fictional roles and allowed to debate without independent evidence, the label conceals rather than explains. The runtime should represent real heterogeneity: a local intent model, a domain retriever, a symbolic planner, an external verifier, a policy gate and a human approver. Their coordination can then be tested as software.

Research hypothesis. A contract-first runtime will outperform prompt-centric agent frameworks on governed task success, even when both use the same underlying models. The advantage should appear in failure localization, repeatability, privilege containment, cost control and the ability to replace one component without retraining the whole system. The claim should be tested on tasks with state, policies and irreversible actions, not only on answer-generation benchmarks.

Conditional outcome. If the hypothesis holds, the durable asset of an agentic organization will be its capability registry and execution receipts rather than its prompt library. The registry will describe what each model or procedure can legitimately do. The receipts will record which versions, evidence, policies and humans participated in an outcome. Models will remain replaceable. Organizational knowledge will reside in contracts, tests, data, formal procedures and governed memories that survive changes in the model market.

PART II

The Agentic Language Stack

Agentic language work is examined as a sequence of interpretations, transformations, formalizations, actions, memories and checks rather than as one undifferentiated prompt.

CHAPTER 4

Reading the Request: Intent, Routing, Ambiguity, and Multilingual Input

The first failure of an agent often occurs before anything recognizable as reasoning begins. A request is assigned to the wrong operation, a prohibition is treated as background information, a reference is resolved to the wrong object, or a clarification that should have been asked is silently invented. These errors are especially dangerous because every later step may be internally coherent. Better planning cannot rescue a misread goal.

Intent detection is frequently described as a solved classification problem. In bounded applications it can be close to one. A banking assistant may distinguish balance inquiry, transfer preparation, card replacement and fraud reporting. A laboratory assistant may distinguish literature search, protocol comparison, dataset analysis and manuscript revision. Yet real requests contain multiple intents, temporal constraints, exceptions, indirect speech and organizational context. “Check whether the invoice can be paid after legal confirms the amendment, but do not block the month-end report” contains a conditional action, a dependency, a prohibition and a timing requirement. The useful output is not one label. It is a structured task hypothesis with unresolved fields.

Established practice. Small encoders and classifiers remain strong choices for language identification, topic routing, named-entity recognition and closed intent sets. Compact generative models become attractive when the label space is compositional or when the output must include spans, arguments and relationships. The two approaches should not be treated ideologically. A classifier is easier to calibrate for a stable taxonomy. A generator can express new combinations but creates a larger error surface. Hybrid systems can use a classifier for coarse routing and a compact

generator for argument extraction, with a general model reserved for ambiguous or novel requests.

The relevant research unit is the **intent contract**. It should preserve the requested outcome, explicit exclusions, entities, temporal conditions, data scope, expected output form and uncertainty. It should distinguish user-stated facts from inferred context. It should carry source spans so later components can trace important constraints back to the language that introduced them. This is more informative than a hidden reasoning trace and more useful than a confidence score detached from the interpretation.

Ambiguity is not one problem. Lexical ambiguity concerns words with several senses. Referential ambiguity concerns which person, document or system a phrase denotes. Scope ambiguity concerns which clause a negation, condition or modifier affects. Procedural ambiguity concerns what organizational process the user assumes. Epistemic ambiguity concerns whether the user is asserting a fact, asking for verification or expressing a hypothesis. Risk ambiguity concerns whether an apparently routine request could trigger consequential action. Different ambiguities need different resolutions. Retrieval can resolve a document version. A user may need to resolve a business intent. A policy engine may determine that no interpretation authorizes the requested action.

Small models are well suited to detecting known ambiguity patterns when examples are available. They are less trustworthy when asked to settle ambiguity from broad world knowledge. This suggests an asymmetry: local components should be trained aggressively to recognize uncertainty and conservatively to resolve it. A model that says “I cannot distinguish supplier policy A from the revised policy B” is useful if the runtime can ask a targeted question. A model that chooses B because it is semantically plausible can contaminate the entire workflow.

Multilingual input adds another dimension. Translation is not always the first step. Names, legal terms, product codes, formulas and mixed-language documents may be damaged by indiscriminate translation. A robust intake layer should detect language by span where necessary, preserve protected terminology, normalize encoding and punctuation, and identify whether the downstream capability supports the source language

directly. For high-volume language pairs, specialized translation models may provide lower latency and more predictable terminology. For rare languages, code-switching or context-sensitive interpretation, a broader model may be needed. Grammatical Framework and controlled-language traditions illustrate a complementary approach in which multilingual surface forms map to a shared abstract representation, though coverage must be engineered explicitly. [85, 84]

Industry signal. On-device model families increasingly advertise multilingual, multimodal and tool-facing capabilities because intake happens where users interact with devices. Apple’s model architecture emphasizes integration with operating-system experiences and privacy-preserving placement. Gemma 3n and Ministral variants target edge use with multimodality and multilingual support. These claims indicate product priorities, but they do not establish equal competence across languages or domains. A system should maintain per-language validation rather than inherit a global “multilingual” label from a model card. [18, 16, 19]

Routing begins at intake but should not end there. A request can be routed to a domain, a workflow, a model or a human. Later states may require different decisions. The initial request may be easy to classify, while an unexpected tool result creates a novel interpretation problem. The router should therefore be stateful and capable of escalating mid-trajectory. FrugalGPT, RouteLLM, RouterBench and later routing studies formalize cost-quality trade-offs and model complementarity. The 2026 LLMRouterBench analysis is particularly sobering: complementarity exists, but many sophisticated routers fail to consistently beat simple baselines under unified evaluation, and a large gap remains to an oracle. [45, 46, 47, 48]

That result yields a principle. Routing is not a clerical prelude to intelligence; it is a prediction problem about competence under a specific state. A router must estimate not merely which model gives the best average answer, but which component can satisfy the current contract. Useful features may include recognized intent, language, document type, tool availability, novelty, context length, sensitivity, required formalism,

verifier availability and historical failure patterns. Model confidence can contribute, but should not be the only signal.

A practical architecture separates **hard routing rules** from **learned routing preferences**. Hard rules enforce data residency, licensing, privacy, safety and capability constraints. A confidential document may be prohibited from leaving a local environment even if a cloud model would be stronger. A medical action may require a validated domain workflow. A symbolic planner may be mandatory when explicit preconditions are available. Within the permitted set, a learned router can choose among models based on expected quality, latency and cost. This prevents optimization from overriding governance.

Speech and multimodal requests require the same discipline. A voice interface introduces transcription uncertainty, speaker attribution and prosody. A screenshot introduces spatial grounding and potentially untrusted embedded text. A graphical-interface agent must distinguish labels from commands and visible content from system policy. Compact multimodal models can serve as perception components, but successful grounding does not imply reliable long-horizon action. The gap between recognizing an interface element and completing a workflow is a recurrent signal that perception, state tracking and control must be evaluated separately.

Research judgement. The intake layer should optimize preservation before interpretation. It should retain the original request, mark transformations, attach source spans, preserve uncertainty and avoid irreversible normalization. A later component may need the original wording to resolve a dispute or detect that a negation was lost. This is especially important when small specialized models are chained: each transformation can make the next task easier while also erasing nuance.

Research hypothesis. A layered intake system combining deterministic preprocessing, compact classifiers, compact structured extraction and explicit ambiguity detection can handle most recurrent enterprise requests while reducing calls to general models. Its success should be measured by downstream task correctness and constraint preservation, not intent accuracy alone. The hypothesis fails if local routing saves

model cost but increases silent misinterpretation or clarification burden.

What present evidence permits us to say. Compact models are credible for bounded routing and extraction. Model pools contain complementary capabilities. Current routers do not reliably identify the best model in every state. Multilingual and multimodal labels conceal uneven coverage. Clarification and abstention are therefore essential system functions, not signs of model weakness.

Conditional outcome. If intake contracts become reliable, agent systems gain a stable semantic front end. Downstream tools no longer need to reinterpret the original request independently. Training data can be organized around real ambiguity classes. Errors can be traced to a source span and a particular mapping decision. The result would be less conversationally impressive than an unconstrained assistant, but substantially more useful for controlled automation.

CHAPTER 5

Language Transformations as Agent Infrastructure

Much of an agent's work is neither reasoning in the grand sense nor external action. It is controlled transformation of language. Documents are translated, normalized, segmented, classified, summarized, compared, redacted, de-identified, rewritten for an audience, converted into claims, or aligned with templates. These operations are often treated as secondary because large models perform them fluently. In production systems they deserve first-class status because they dominate volume and because small semantic losses propagate.

A transformation should be defined by what must change and what must remain invariant. Translation changes language while preserving meaning, modality, names, quantities and domain terminology. Summarization reduces length while preserving decisions, exceptions, uncertainty and evidence coverage. Redaction removes sensitive content while preserving enough structure for the downstream task. Normalization regularizes dates, units or terminology without turning disputed language into settled facts. Rewriting changes register or style without strengthening claims. The invariant is the real specification.

Consider a scientific-review agent. It receives a paper, extracts claims, identifies methods, links claims to evidence, summarizes limitations and prepares questions. A generic summary may be readable yet remove the exact population, confidence interval, negative result or stated limitation that determines scientific meaning. A specialized transformation model can be valuable if it is trained and checked against a schema that preserves these items. The goal is not a better abstract. It is an intermediate representation that later reviewers can trust to be incomplete in known ways rather than complete in appearance.

Established practice. Machine translation, information extraction, embeddings, reranking, sentiment and document classification have long supported specialized models. Generative SLMs expand the range of controllable transformations because instructions can specify output schemas and local context. Compact Gemma, Phi, Qwen, Llama, SmolLM and Ministral models can be adapted for such tasks, while dedicated families such as TranslateGemma, EmbeddingGemma, ShieldGemma and MedGemma illustrate product-level specialization. [15, 8, 9, 11, 6, 19, 72]

The strongest case for specialization appears when the organization can define a transformation profile. A legal profile may preserve clause numbering, defined terms, obligations, exceptions and jurisdiction. A clinical profile may preserve negation, temporality, experiencer and measurement units. A financial profile may preserve currency, period, accounting basis and materiality qualifiers. A scientific profile may preserve claim type, evidence source, uncertainty and experimental conditions. These profiles make errors observable and create data for targeted adaptation.

Translation reveals why “small” and “specialized” are independent. A small multilingual general model may support many languages with variable depth. A larger translation specialist may provide better terminology for a narrow pair. A very small local model may be preferred for privacy or offline use despite weaker performance. The decision depends on domain, language pair, sentence complexity, document length and the availability of terminology constraints or human review. A model card’s average multilingual score cannot resolve these choices.

Summarization should be decomposed into selection and realization. Selection determines which claims, facts and relations must appear. Realization turns the selected material into coherent text. General models are strong at realization, which can hide weak selection. An S*LM architecture can use extractive or compact models to identify required items, deterministic checks to confirm coverage, and a fluent model to compose the final prose. The composition may still introduce implications not present in the source, so claim-level checking remains necessary.

This distinction matters for “smoothing,” a common property of generated prose. Models tend to resolve local conflict, regularize terminology and create a coherent narrative. That can improve readability and remove accidental noise. It can also erase disagreement, soften uncertainty or transform inconsistent evidence into an apparently unified conclusion. A transformation profile should state whether contradictions must be preserved, whether competing terminology may be harmonized, and whether causal or normative connections may be introduced. The safest default for scientific and legal work is to preserve unresolved conflict explicitly.

Redaction and privacy-preserving transformation require a different contract. Removing names is insufficient when rare combinations of attributes remain identifying. A model can also regenerate sensitive information from context. Local execution reduces exposure to external services but does not solve access control, logging or retention. A robust pipeline should classify data, minimize context, use deterministic detectors where possible, record what was removed, and test whether the transformed artifact still satisfies the intended privacy objective. The model is one component of a data-governance process.

Industry signal. The proliferation of embedding, guardian, translation and medical variants shows that vendors see value in models that occupy one stage of an application rather than the whole assistant. IBM’s model family similarly separates language, speech, vision, embedding and guardian capabilities. This is more than packaging. It reflects different training data, architectures, output contracts and deployment requirements. The lesson for system designers is to evaluate component roles directly rather than infer them from the capabilities of a flagship general model. [15, 20]

Academic signal. Research on distillation and step-by-step supervision shows that smaller models can inherit useful behavior when training targets expose intermediate rationales or task structure. Tool-use data generation shows a related phenomenon: when examples are executable and verifiable, specialization can be data efficient. These results support task-specific transformation models, but they also warn that student models inherit teacher errors and synthetic-data biases. Human-curated edge cases and independent checks remain necessary. [67, 29, 32]

Transformation chains create a cumulative-risk problem. Suppose a document is translated, summarized, classified and converted into a formal request. Even if each stage is usually correct, errors can interact. A mistranslated negation changes the summary; the summary changes the intent class; the wrong class selects an inappropriate formalizer. The system should therefore minimize unnecessary transformations and retain lineage. Where possible, later checks should consult the original source rather than only the previous model's output.

A useful pattern is **canonical plus evidence**. The transformation produces a canonical object and links each field to source spans. A contract extractor returns normalized parties and obligations together with the clauses that support them. A scientific claim extractor returns the claim, population, intervention, outcome and uncertainty together with exact passages. A translation system records protected terms and unresolved alternatives. This lets later components work on a simpler representation without losing access to the linguistic evidence.

Another pattern is **dual realization**. The system stores a structured result as the authoritative artifact and generates prose as a view. If the prose changes, the structured object remains stable. If a reader questions a sentence, the system can show the fields and sources that generated it. This does not make the structured object infallible, but it prevents fluent text from becoming the only record of interpretation.

Research judgement. Language transformations should be treated like compilers with declared invariants, not like stylistic conveniences. Their tests should include semantic mutations: negation, quantities, units, named entities, modality, temporal order, exceptions and conflicting evidence. Ordinary similarity metrics are insufficient because a sentence can remain semantically close while changing the condition that controls an action.

Research hypothesis. For recurrent workflows, source-linked compact models should match general models at lower cost under invariance tests and escalation; maintenance may erase the gain. **Conditional outcome.** Reusable schemas and validators become durable assets.

CHAPTER 6

From Natural Language to Formal Objects

The most consequential boundary in an S*LM system is the transition from language to an executable or checkable object. A natural-language statement can tolerate ambiguity, implication and context. A schema, query, program, logical theory or plan must make choices. Formalization is therefore not a clerical conversion. It is an epistemic act that determines what the system will treat as true, relevant and actionable.

The target may be modest. A model converts “show overdue invoices above ten thousand euros” into a typed function call with a threshold, currency and status filter. The target may be a database query, a controlled natural-language statement, a regular expression, a knowledge-graph query, a set of constraints, a PDDL planning problem, a theorem-prover declaration or a code patch. These objects vary radically in expressive power and assurance. They should not be grouped under “structured output” as though syntax were the main difficulty.

A useful formalization pipeline has four stages. It identifies the target vocabulary and available operations. It maps source phrases to entities, predicates, parameters and relations. It constructs a candidate artifact. It validates syntax, types, examples, source correspondence and domain constraints. Only after these stages may an executor or solver use the artifact. A compact model can perform one or several stages, but the validation burden grows with semantic distance from the source.

Established practice. Grammar-constrained decoding can ensure that generated outputs conform to JSON, regular languages or other formal grammars. XGrammar and Outlines implement efficient structured generation across model runtimes. JSONSchemaBench and related work show that schema coverage and implementation details matter, and that a valid object is only the beginning of downstream correctness. [42, 43, 44]

The first design insight is that grammar should be as narrow as the operation permits. If the agent has five approved tools, the model should not emit arbitrary code to invoke them. If a database offers a reviewed query API, the model should populate parameters rather than write unrestricted SQL. If a planner has a fixed domain model, the model should identify objects and goals rather than invent action semantics. Narrow targets reduce search and attack surface. They also make small models more competitive because the output distribution is constrained.

The second insight is that semantic choices must remain visible. When “recent” becomes thirty days, when “high risk” becomes a threshold, or when “supplier” maps to one of several entity types, the conversion should record the rule or assumption. Silent defaults are convenient until they are wrong. A formal artifact should carry source spans, terminology versions, unresolved alternatives and any human-approved conventions used in translation.

Controlled natural languages occupy an intermediate position. Attempto Controlled English and related systems restrict grammar so sentences map more predictably to logic. Grammatical Framework separates abstract syntax from multilingual concrete syntax. Domain-specific CNLs can make policies, procedures or scientific claims more reviewable without demanding direct authoring in a theorem prover. Their limitation is coverage: the more natural and expressive the language becomes, the more ambiguity returns; the more controlled it becomes, the more training and author discipline it requires. [84, 85]

A compact language model can make CNL practical by translating unrestricted input into candidate controlled statements and explaining diagnostics. The controlled form can then be parsed deterministically. This is a promising role because the model handles paraphrase and local ambiguity while the grammar defines the accepted surface. The risk is semantic laundering: a fluent translator may produce a valid CNL sentence that is easier to execute but subtly different from the source. The system must compare the candidate against examples, source spans and counterfactual mutations rather than accept compilation as proof of fidelity.

Database formalization illustrates the value of layered targets. Natural language can first become a domain intent such as “retrieve overdue invoices,” then a typed query object, then parameterized SQL generated by reviewed code. This preserves database policy and makes the model independent of physical schema changes. Direct text-to-SQL may be justified in exploratory analytics, but consequential write operations should pass through narrower procedures, row-level access controls and transaction boundaries.

Planning formalization is harder. PDDL requires objects, predicates, initial state, goals and action semantics. LLM+P showed the value of letting a language model translate a problem and a classical planner perform search. PlanBench and related analyses demonstrated that language models often produce plausible plans without reliable state tracking or respect for preconditions. The hybrid architecture improves combinatorial search, but its success still depends on an accurate domain and problem encoding. A planner cannot detect an omitted real-world hazard unless the formalization represents it. [38, 40, 83]

Logical reasoning systems expose the same boundary. Logic-LM translates natural-language problems into symbolic forms, invokes a solver and uses diagnostic feedback to repair errors. LINC combines language models with first-order logic provers. SatLM uses satisfiability-oriented declarative representations. These systems provide stronger evidence that external reasoning can improve results on formalizable tasks. They do not show that arbitrary language can be converted into faithful logic automatically. The translation step remains the learned and fallible component. [35, 36, 37, 81]

Program generation adds execution semantics and security risk. Code can express operations more flexibly than a schema, but arbitrary generated code expands authority enormously. The S*LM principle is to use the narrowest executable representation that covers the need. A calculation may be a tested library call. A data transformation may be a declarative pipeline. A policy may be a rule set. Generated code should run in a sandbox, under resource limits, with restricted dependencies and tests derived from the task contract. Even successful tests establish behavior only over the tested envelope.

Formalization can also move in the opposite direction. After a solver, database or program produces a result, a model explains it in natural language. This realization should be constrained by the actual artifact and evidence. A model should not write an independent persuasive story from the final answer. It should render the selected records, derivation, plan or test results, preserving uncertainty and limitations. Bidirectional translation makes both directions auditable: language to formal object, formal result back to language.

Research judgement. Formalization is often the right place for a specialized model, but the wrong place for unqualified autonomy. It has a stable target language and can benefit greatly from narrow training. At the same time, its errors can be structurally valid and consequential. The architecture should optimize for explicit alternatives, diagnostic feedback and abstention rather than force every sentence into one formal interpretation.

Research hypothesis. Compact verifier-aware models can become reliable semantic compilers for bounded domains when trained not only on successful pairs but on compiler errors, counterexamples, ambiguous cases and source-fidelity checks. The important output is not merely a formal artifact; it is an artifact plus a validation envelope and unresolved assumptions. This hypothesis is stronger than ordinary structured generation and weaker than general autoformalization.

What current work signals. Constrained decoding is mature enough to treat syntax as an engineering problem. External solvers and planners can supply exact search and diagnostics. Language-to-formal translation remains the bottleneck. Research should therefore stop using parser acceptance as the primary success criterion and evaluate whether formalization preserves the distinctions that matter to downstream decisions.

Conditional outcome. If verifier-aware semantic compilation succeeds, policies can be checked, plans simulated, queries parameterized and explanations tied to executed artifacts. Compact models would become adaptable interfaces to formal systems while preserving the boundary between proposal and proof.

CHAPTER 7

Planning, Tool Use, and Controlled Action

Function calling is often presented as the point at which a language model becomes an agent. In reality it is only one boundary in a control system. A model must recognize whether a tool is needed, retrieve the relevant tool from a potentially large catalogue, select it, construct arguments, interpret the result, update state, decide what follows and respect policy throughout the trajectory. A system can be excellent at isolated function syntax and poor at accomplishing tasks.

The simplest useful case is a closed action space. A mobile assistant may control alarms, timers, media, settings and a few applications. An enterprise assistant may expose approved procedures for creating a report, retrieving a record or preparing a workflow change. In such settings the tool description, parameter schema and examples form a compact action language. FunctionGemma, Hammer and related work suggest that very small models can learn this language when the application distribution is narrow and the model is adapted to it. [13, 14, 31]

APIGen, xLAM, ToolACE, ToolLLM and Gorilla address larger tool spaces through synthetic data, verification, retrieval and trajectory construction. Their main contribution to the S*LM programme is methodological. Tool-use data should not be generic dialogue with function names inserted. It should represent valid and invalid arguments, irrelevant tools, abstention, multi-turn state, tool errors and executable outcomes. Where possible, calls should be run or checked. A small action model benefits disproportionately from this discipline because it has less capacity to compensate for noisy interface descriptions. [29, 30, 32, 33, 27, 28]

Established practice. A production tool layer should validate schemas, authenticate outside the model, enforce rate and resource limits, separate read from write operations, and capture outputs as data rather than

instructions. The model proposes calls; application code executes them. Consequential tools should have explicit approval and rollback semantics. These practices are ordinary security and software engineering, but agent frameworks often weaken them by presenting every capability inside one conversational loop.

Tool retrieval is a specialization problem of its own. Large catalogues overload context and increase confusion between similar functions. A compact retriever or embedding model can first select a small candidate set. A second model can choose among those candidates and fill arguments. Policy can filter the set before either model sees it. This staged design makes errors interpretable: retrieval missed the correct tool, selection chose the wrong candidate, or argument construction failed. A monolithic prompt obscures these distinctions.

Planning is different from tool selection. A plan represents dependencies and future actions under a model of state. Language models are good at proposing plausible decompositions, especially where steps are described informally. They are less reliable at exhaustive search, exact resource accounting and long sequences with interacting preconditions. ReAct improved the practical loop by letting observations inform subsequent reasoning, but a long trace can accumulate irrelevant context and hidden assumptions. Classical planners, workflow engines and explicit state machines remain superior when the domain has stable actions and state transitions. [25, 38, 40, 83]

The best architecture is often hierarchical. A general or compact planning model identifies a coarse strategy and unresolved choices. Domain procedures or a symbolic planner expand stable portions. An action model handles local tool calls. The runtime observes results and revises only the affected part of the plan. This is less glamorous than an autonomous stream of thought, but it avoids asking one model to hold the entire world state in text.

A useful distinction is between **procedural novelty** and **parametric novelty**. Parametric novelty changes values inside a known procedure: a new supplier, date range or dataset. Small specialized models and deterministic workflows can usually handle it. Procedural novelty changes

the structure of the work: a new regulation, tool or scientific method. It may justify a general model, coding agent or human designer. Systems waste resources when they treat every new parameter as a novel reasoning problem; they become unsafe when they treat genuinely new procedures as routine parameters.

State should be represented explicitly. After a tool call, the system should know which facts were observed, which actions succeeded, which failed, which effects are reversible and what version of the external system was involved. A language transcript is a poor substitute. It encourages the model to infer state from prose and makes retries dangerous. Typed state enables idempotency keys, transactions, checkpoints and compensation actions. It also lets a small model receive only the local facts required for its next decision.

Security boundary. Tool outputs are untrusted input. A webpage, email, repository or document can contain language designed to redirect the agent. InjecAgent and the OWASP agentic risk taxonomy demonstrate why indirect prompt injection is a system problem rather than a prompt-writing inconvenience. The runtime must distinguish data from control, restrict tool authority and validate proposed side effects independently of retrieved text. Progent's policy layer is one example of moving privilege decisions outside model generation. [59, 58, 60]

The action gate should evaluate more than syntax. It should ask whether the tool is permitted for this identity, whether the arguments fall within scope, whether preconditions are satisfied, whether the action conflicts with the user's exclusions, whether sensitive data would leave an approved environment, whether the action is reversible and whether human confirmation is required. Some of these checks are ordinary code; others can be expressed in rules or constraints. A language model may summarize the decision, but it should not be the sole judge of its own authority.

Verification can occur before and after action. Pre-execution checks validate plans, policies and parameters. Simulation may reveal a conflict. Post-execution checks compare observed effects with expected effects and

detect partial failure. A planner may then repair the plan or escalate. LLM-Modulo's broader lesson applies here: generation can remain flexible if hard and soft critics are independent and if rejection produces useful diagnostics. [39]

Industry signal. Compact models are increasingly marketed with built-in function calling, long context and edge deployment. The meaningful trend is not the phrase “agentic model,” which is often marketing. It is the availability of model cards, templates and runtimes that make structured calls a supported product path. IBM emphasizes tool calling and predictable token use for enterprise workloads. Google provides a model whose explicit purpose is customized local function calling. Apple exposes constrained tool calls within an on-device framework. Mistral distributes compact multimodal variants for edge systems. [13, 17, 20, 19]

Academic signal. Tool-use research has moved from demonstrating that models can call APIs to studying verifiable data, irrelevant tools, multi-turn trajectories and orchestration. ToolOrchestra makes the small model an orchestrator over stronger models and tools rather than the sole problem solver. This is a consequential inversion: the compact model's intelligence lies in allocation, not in internally reproducing every capability. The evidence remains task-dependent, but it broadens the role of small models from cheap executors to learned control policies. [34]

Research judgement. Tool evaluation should measure the completed, policy-compliant state transition, not isolated function syntax. Correct calls remain unreliable when they misinterpret intent, violate policy, repeat effects or fail to recover.

Research hypothesis. Specialized action models should handle high-volume decisions when retrieval, typed state, verification and escalation are supplied; the claim weakens as tools or procedures change rapidly. **Conditional outcome.** Success would yield a governed runtime in which compact models propose actions, workflows manage dependencies, policy gates control authority, and larger models or humans handle novelty.

CHAPTER 8

Memory, Retrieval, Verification, Recovery, and Escalation

An agent that cannot preserve state cannot work across time. An agent that preserves everything cannot reliably find what matters and may retain information it should forget. Memory is therefore not a larger prompt. It is a governed data system whose contents have different origins, lifetimes, authorities and uses.

A practical memory architecture distinguishes working state, factual memory, experiential memory and procedural memory. Working state contains the current task, intermediate values and pending actions. Factual memory contains claims about users, organizations or the environment. Experiential memory contains prior trajectories, outcomes and failure patterns. Procedural memory contains reusable instructions, tools, schemas and policies. These categories may share infrastructure, but they require different write rules. A tool observation may update working state immediately. A model inference should not become a durable fact without validation. A failed trajectory may be valuable experience even though its conclusions are rejected.

The 2025–2026 memory literature explicitly moves beyond the long-term versus short-term dichotomy. The broad survey “Memory in the Age of AI Agents” organizes memory by form, function and dynamics. A later data-management study decomposes systems into representation and storage, extraction, retrieval and routing, and maintenance, finding no single architecture that dominates across workloads. This is exactly the level at which S*LM design should operate: memory choices depend on the task bottleneck rather than on a generic vector-store recipe. [53, 54]

Established practice. Retrieval systems can reduce the need to place entire corpora or histories in context. Databases support versioning, transactions, access control and structured queries. Checkpointed

workflows support pause, resume and rollback. Embedding and reranking models can be compact and locally deployed. These capabilities are mature individually. The open problem is their semantic integration: what should be written, how should conflicts be represented, and when is a retrieved item authoritative enough to influence action?

A memory write should include provenance. “The supplier is restricted” could originate from a user statement, an official policy, an inferred classification or an earlier model summary. These are not equivalent. The stored object should identify source, time, model or tool version, confidence where meaningful, validation status, sensitivity and expiry. A later agent can then decide whether the item is admissible for a particular purpose. Provenance does not prove truth, but it prevents origin from disappearing behind fluent restatement.

Memory maintenance is as important as retrieval. Facts become stale. Policies are superseded. User preferences conflict. Derived summaries lose the evidence that once supported them. Global re-summarization is expensive and can introduce drift. Localized updates and versioned objects are often safer. The database-oriented memory study’s finding that workload alignment matters suggests that agent memory should expose maintenance policies rather than rely on one universal consolidation algorithm. [54]

Small models can occupy several memory roles. An extractor can decide which parts of a conversation are candidates for durable storage. A classifier can assign type, sensitivity and expiry. An embedding model can index content. A reranker can select task-relevant items. A compact summarizer can create a view for the current operation. A contradiction detector can flag incompatible claims. Yet none of these models should silently grant durable authority to its own output. The runtime must distinguish candidate memory from promoted memory.

Retrieval is a form of routing. It predicts which information will help a component satisfy its contract. Similarity is only one signal. Recency, authority, dependency, task stage, access permission and past utility may matter more. A policy question should prefer the latest approved rule over a semantically similar obsolete draft. A scientific review should preserve

contradictory studies rather than retrieve only the dominant cluster. A personal assistant should not infer that a transient preference is permanent.

Verification is the second source of memory. Tests, solvers, database constraints and external observations create stronger artifacts than unconstrained model judgements. A code agent can store that a test passed under a particular commit and environment. A planning agent can store a counterexample. A research agent can store that a dataset was unavailable or that a claimed analysis could not be reproduced. Negative and inconclusive results are valuable because they prevent repeated dead ends. They must remain distinct from accepted facts.

A verifier should itself have a contract. A schema validator establishes syntax and types, not semantic truth. A unit test establishes behavior over selected cases. A theorem prover establishes a result under formal assumptions. A retrieval-based critic establishes correspondence to available sources only to the extent that retrieval is complete and the source is credible. A model judge adds another probabilistic opinion. Systems become misleading when these different warrants are collapsed into one confidence score.

Recovery is where agentic reliability becomes visible. Tools time out, APIs change, credentials expire, files conflict and models produce invalid outputs. A robust runtime classifies failures. Transient failures may justify retry. Invalid arguments may justify repair. Contradictory evidence may justify clarification. A policy violation should not be repaired by rephrasing until the gate passes. An irreversible partial action may require human intervention. Recovery rules should be explicit because unconstrained retry loops can amplify cost and damage.

Agent reliability research argues that average success hides consistency, robustness, predictability and severity. This is especially relevant for small-model systems. A compact model may succeed frequently on routine states but fail sharply outside distribution. That can still be useful if the runtime detects the boundary and escalates. It is dangerous if the model fails fluently and the system treats low average cost as sufficient. [61, 49]

Escalation should be a designed operation, not the residue of failure. It may send the task to a larger model, a domain specialist, a symbolic procedure, another data source or a human. The escalation package should include the local interpretation, evidence, failed checks and unresolved questions, not the entire unfiltered transcript. This makes the expensive component more effective and preserves the work already performed.

A good escalation policy balances false confidence and unnecessary expense. Over-escalation defeats the economic purpose of specialization. Under-escalation creates silent failure. Routing studies show that this frontier is difficult to learn from generic preference data. Better signals may come from the contracts themselves: missing required fields, unfamiliar entities, schema repair counts, contradiction density, solver failure, novelty against the validated dataset, or high consequence. [46, 48]

Research judgement. Memory and escalation are what make specialization safe. A narrow model need not solve every case if it can recognize when its evidence is insufficient, preserve its partial result and hand off cleanly. Conversely, a powerful model without governed memory may repeatedly rediscover context, overwrite facts with summaries and act on stale assumptions.

Research hypothesis. Agent memory should be evaluated as a data system with semantic write policies, not only by whether a downstream benchmark score improves. A memory architecture that preserves provenance, conflict, expiry and validation status should reduce long-horizon failure even if it retrieves fewer superficially relevant passages. Compact models can manage much of this lifecycle when their outputs are treated as proposals within typed policies.

Conditional outcome. If this programme succeeds, memory becomes the durable substrate of agentic systems. Models can be upgraded without erasing organizational state. Failures become reusable constraints. Privacy policies can govern retention and deletion. Escalation becomes selective rather than panicked. The agent's competence will depend less on how much text fits in a context window and more on whether the runtime knows what should be remembered, trusted, forgotten and revisited.

CHAPTER 9

Multi-Agent Systems Without Role-Play

Multi-agent systems are attractive because complex organizations already divide work among specialists. The analogy becomes misleading when it is implemented as several copies of the same model receiving theatrical job titles and exchanging unconstrained prose. More agents do not automatically create independent expertise. They can create duplicated errors, communication overhead and the illusion of review.

Real heterogeneity has a technical basis. One component may have access to a private database, another to the public web, another to a theorem prover, another to a code sandbox and another to human approval. Models may differ in size, domain training, modality, cost and deployment location. Memories and authorities may be separated. In this setting, an agent label can clarify ownership of a contract. A literature-search agent and a statistical-verification agent are different because they operate over different evidence and tools, not because their prompts describe different personalities.

Academic evidence. MAST analyzes empirically grounded multi-agent failure modes, including problems in task specification, inter-agent misalignment and verification. The broader “AI Agents That Matter” critique emphasizes cost, reproducibility and baseline quality. These works warn that multi-agent gains can disappear when compared under controlled budgets or when coordination failures are counted. [52, 49]

Anthropic’s engineering account of a multi-agent research system presents a more favorable but equally instructive case. Parallel subagents can search different parts of a problem while an orchestrator integrates results. The value comes from parallel exploration and context separation, not from fictional debate. The architecture also incurs much higher token use and requires careful prompting, tool design and evaluation. This is an

industry report about one system, not a universal result, but it illustrates conditions under which specialization and parallelism can be useful. [51]

The first multi-agent design question is whether work is genuinely decomposable. Independent literature searches can run in parallel. Separate code changes to unrelated modules may run in parallel. A plan with tightly coupled state transitions may not. Parallel agents can create conflicting assumptions when the task requires a shared evolving world model. The orchestrator must know which outputs commute, which depend on earlier results and which require serialization.

The second question is whether agents provide independent evidence. Two instances of the same model prompted differently may disagree, but their errors remain correlated through training data and architecture. Majority voting can amplify a shared misconception. Independence improves when agents use different models, tools, data sources or verification methods. A symbolic checker offers a different failure mode from a generative critic. A human domain expert offers contextual judgement that another model may lack. The objective is diversity of warrant, not diversity of tone.

The third question is how communication is represented. Free-form messages are flexible but expensive and ambiguous. Typed artifacts reduce communication cost and preserve meaning. A search agent can return claims with sources and coverage notes. A code agent can return a patch, tests and execution logs. A planner can return a dependency graph. A critic can return failed invariants and counterexamples. The orchestrator should not need to infer every subagent's result from prose.

Small models are plausible workers when the work package is narrow. A compact model can classify documents, extract structured fields, call a defined tool or verify a local property. A larger orchestrator may allocate tasks and synthesize uncertain results. ToolOrchestra explores the inverse possibility: a small orchestrator can coordinate stronger models and tools. Both arrangements are credible because orchestration and execution have different information requirements. The correct choice depends on whether the controller must understand broad task semantics or mainly select among well-described capabilities. [34]

A hierarchical architecture usually scales better than all-to-all conversation. A coordinator assigns contracts, receives typed results and maintains global state. Workers communicate through the coordinator or a shared artifact store. This reduces context duplication and makes authority explicit. It also creates a central bottleneck and single point of interpretive failure. The coordinator may therefore need independent plan checks, state validators or human oversight for high-consequence tasks.

Multi-agent memory introduces further complexity. Shared memory can improve coordination but spreads errors and sensitive data. Private memory preserves specialization but can create incompatible world views. A practical system should distinguish shared verified state from local working notes. Promotion into shared memory should require a contract and provenance. The memory literature's emphasis on lifecycle and maintenance becomes even more important when multiple agents write concurrently. [53, 54]

Verification roles deserve special scrutiny. Calling one model a critic does not make it independent. A critic should have access to criteria and evidence that differ from the generator's. For code, this may be tests and static analysis. For planning, it may be a simulator or formal constraints. For scientific work, it may be source-level checks, statistical recomputation or methodological rules. Model-based criticism can still be useful for open-ended issues, but its uncertainty and possible collusion with the generator should remain visible.

Coordination cost is not merely token cost. It includes latency, state reconciliation, duplicate retrieval, error propagation and debugging complexity. A multi-agent design should earn this cost through parallelism, specialization, security separation or independent verification. If one model with two tool calls performs the task as reliably, the multi-agent system is unnecessary. Anthropic's guidance to begin with the simplest architecture and add agentic complexity only when it improves outcomes remains a sound industrial principle. [50]

The same principle applies to human-agent teams. Humans should not be used as generic exception handlers after the system has exhausted its budget. Their role should be aligned with judgement, authority and

responsibility. An agent can collect evidence and frame the unresolved choice. The human can decide which trade-off is legitimate, approve an irreversible action or revise the task definition. The handoff should make the machine's limits visible rather than overwhelm the reviewer with a transcript.

Research judgement. Multi-agent systems are most defensible when each agent corresponds to a different capability contract, information boundary or authority. Role-play is not specialization. Debate is not verification. More messages are not more reasoning. The architecture should minimize communication and maximize the semantic quality of exchanged artifacts.

Research hypothesis. Heterogeneous teams of compact specialists, deterministic tools and selective general models can outperform monolithic agents on complex workflows when decomposition is stable and outputs are typed. The benefit should disappear or reverse when subtasks are tightly coupled, workers share the same failure mode or the orchestrator cannot validate their outputs. This hypothesis requires cost-controlled and failure-aware evaluation rather than aggregate success alone.

Conditional outcome. If these conditions are understood, multi-agent design can become an engineering discipline rather than a metaphor. Teams will be assembled around evidence, tools, privacy and authority. Small models will provide scalable labor where tasks are repetitive. General models will handle synthesis and novelty. Symbolic components will supply independent checks. Humans will govern consequential commitments. The resulting system may contain many agents, but it will behave more like a well-designed distributed program than a simulated meeting.

PART III

Trust, Deployment, and Economics

Efficiency is valuable only when the resulting system remains governable, robust, private where required, and economically sound at the level of successful tasks.

CHAPTER 10

Trustworthy AI by Architecture

Trustworthiness is sometimes discussed as though it were a personality that can be trained into a model. Agentic systems make the limitation of that view obvious. A courteous, aligned and seemingly cautious model may still read data it should not access, call a tool with excessive privilege, rely on a poisoned document, overwrite valid state or produce an explanation disconnected from what actually happened. Trustworthiness belongs to the relationship among model behavior, data, tools, policies, users and institutions.

Small, specialized and symbolic components change this relationship, but not uniformly. A smaller model can run locally and expose less data to external providers. It may also have weaker instruction following and narrower robustness margins. A specialized model can be evaluated deeply on a defined distribution. It may fail abruptly when the distribution shifts. A symbolic checker can enforce an explicit constraint. It cannot detect harms omitted from the formalization. The research question is therefore not whether S*LM systems are inherently safer. It is which trust properties become easier or harder under each decomposition.

Alignment. In a bounded agent, alignment should be translated into operational contracts. What outcomes are permitted? Which instructions have priority? Which data may be used? What must trigger refusal or human approval? A general model's post-training still matters because it shapes interpretation and behavior under uncertainty. Yet stable organizational rules should not live only in weights or system prompts. They should be represented as tool permissions, workflow constraints, validation rules and review gates. NIST's Govern–Map–Measure–Manage framing supports this system view by treating risk as context-dependent and continuous rather than solved at model release. [57]

Specialization can make alignment more testable. A model that only maps requests into ten approved intents can be examined against

adversarial paraphrases, prohibited combinations and abstention cases. A function caller can be tested for unauthorized calls and parameter boundaries. A domain summarizer can be checked for preservation of uncertainty. But narrow validation creates a temptation to overgeneralize. Passing a test suite establishes a validation envelope, not universal alignment. The runtime must detect when the current task falls outside it.

Bias and fairness. Small models do not automatically reduce bias. Distillation can transfer a teacher’s preferences. Domain fine-tuning can amplify the imbalances of a smaller corpus. Quantization and other inference optimizations can change bias in model-specific and unpredictable ways even when average task quality appears stable. A symbolic rule can make a decision procedure transparent while formalizing an unfair policy perfectly. Fairness must therefore be defined for the particular decision, population and harm, then evaluated after adaptation and deployment. [68]

Decomposition can nonetheless improve governance. A hiring-support system can separate text extraction from a policy that prohibits sensitive attributes, a ranking model, subgroup analysis and human decision. Each boundary becomes auditable. The extraction model may still infer proxies, the ranking data may still encode historical discrimination and the human may still over-rely on the result. Architecture does not remove social judgement, but it prevents one opaque score from concealing all stages.

Sycophancy. Models may adapt answers to a user’s expressed belief rather than preserve evidence-based judgement. Research links some sycophantic behavior to preference training, while later reviews warn that the term is operationalized inconsistently and often measured without human perception. The practical risk in agents is broader than agreement in conversation. A planning model may accept an unsafe premise. A reviewer may soften criticism to match the author’s framing. A scientific agent may reinterpret evidence to preserve a favored hypothesis. [55, 56]

An S*LM architecture can reduce this risk by separating evidence collection, evaluation and presentation. A source checker need not know the user’s preferred conclusion. A statistical validator can run on a formal artifact. A critic can receive criteria and evidence without receiving

persuasive framing. A final model can explain disagreement after checks are complete. This does not guarantee independence, especially if all components share the same base model and training data. It does create opportunities for information separation and different warrants.

Hallucination and unsupported claims. Smaller models may hallucinate more under broad tasks, while specialized models can be highly accurate in-domain and brittle outside it. Symbolic execution prevents some classes of invented computation but not invented premises. The appropriate objective is not to eliminate all generative uncertainty. It is to prevent unsupported text from becoming authoritative state. Claims should carry sources or derivations. Unknowns should remain unknown. A failed retrieval should not be converted into a confident universal negative.

Transparency and explanation. A model-generated rationale is not the same as a causal record. A trustworthy runtime first records inputs, retrieved evidence, model and tool versions, formal artifacts, policy decisions, state changes and human approvals. A language model may then summarize this trace for different audiences. This provides structural explanation without claiming to expose the hidden internal reasoning of a neural model. The distinction is important because plausible explanations can increase trust while decreasing understanding.

Privacy. Local inference is one of the clearest benefits of compact models. It can keep raw text, voice, images or organizational data on device or within a controlled environment. Apple, Google and other vendors explicitly develop on-device model and runtime stacks around this advantage. Yet locality is only one control. The system still needs data minimization, access policies, memory retention rules, encryption, audit and deletion. A local model with unrestricted access to every file is not privacy preserving merely because no cloud API is called. [17, 18, 14, 16]

Privacy and auditability can conflict. Detailed traces help reproduce decisions but may retain sensitive data. The architecture should support layered provenance: immutable identifiers and decision metadata where lawful, protected raw artifacts under access control, and explicit records when deletion makes full reproduction impossible. “Perfect memory” is not the goal. Governed memory is.

Security. Agentic authority magnifies ordinary model errors. Prompt injection can turn retrieved data into attempted control. Tool descriptions can be poisoned. Generated code can import malicious dependencies. Memory can retain attacker instructions. Small models may be attractive attack targets because they are deployed widely on devices and may have weaker defenses. Security must therefore be enforced outside the model through capability scoping, sandboxing, input provenance, output validation, rate limits, transaction boundaries and monitoring. InjecAgent, Progent and the OWASP agentic risk taxonomy provide concrete evidence for this system-level approach. [59, 60, 58]

A symbolic policy engine can enforce least privilege, but the policy itself must be correct and maintained. Automatically generated policies create a useful but delicate pattern: a model proposes a policy, deterministic machinery enforces it, and diagnostics support revision. The enforcement can be exact relative to the policy while policy synthesis remains uncertain. This mirrors the central S*LM principle: formal components strengthen the system only when the neural–symbolic boundary remains visible.

Robustness and reliability. Average accuracy is inadequate for agents. The system should measure consistency across runs, sensitivity to harmless perturbations, behavior under missing or conflicting data, recovery from tool failure, and severity of side effects. The 2026 agent-reliability study reports that capability gains have not translated into equally large reliability gains under a multidimensional profile. Compact components should therefore be evaluated for bounded degradation and reliable escalation, not only peak success. [61]

Human factors. A specialized system may look more reliable because its interface is structured and its outputs are concise. This can increase automation bias. Users should see the validation envelope, evidence coverage, unresolved assumptions and whether a result came from deterministic execution or a model judgement. Human approval should be placed where it changes authority, not added as ceremonial clicking after the system has made every substantive decision.

Research judgement. Trust should be allocated as carefully as computation. A model may be trusted to recognize a familiar intent but

not to authorize payment. A solver may be trusted to satisfy encoded constraints but not to define the constraints. A retriever may be trusted to locate documents but not to assess their scientific credibility. A human may be trusted to approve an action but still need calibrated evidence. The architecture should represent these differentiated trusts rather than assign one confidence score to the whole agent.

Research hypothesis. S*LM systems can improve trustworthy AI when decomposition narrows authority, exposes provenance and enables independent checks. The same decomposition can worsen trust when it multiplies unvalidated transformations, hides correlated models behind agent roles or treats formal syntax as semantic proof. The programme should compare both outcomes empirically through threat models, failure injection and human-calibration studies.

Conditional outcome. If the hypothesis is supported, trustworthy agent design will move away from asking whether a model is aligned in the abstract. It will ask whether each component is competent for its role, whether data and authority are scoped, whether formal guarantees match their representations, whether failures are contained and whether humans can understand what remains uncertain. S*LM would then be valuable not because smaller or symbolic systems are morally superior, but because a well-designed division of labour makes trust claims more precise and falsifiable.

CHAPTER 11

Training, Deployment, and the Economics of Successful Work

The economic case for small models is usually expressed in tokens, parameters or accelerator hours. Those measures are useful but incomplete. An agent creates value only when it completes a task correctly, within policy, at acceptable latency, without costly recovery or human rework. A cheap model that requires repeated retries, misroutes sensitive data or generates invalid actions may be more expensive than a stronger model used once. The proper unit is the cost of successful, governed work.

Training cost has several layers. Pretraining from scratch builds a general foundation and demands large data and compute. Continued pretraining adapts vocabulary and knowledge to a domain. Supervised fine-tuning teaches an instruction or output distribution. Preference and reinforcement methods shape behavior. Distillation transfers outputs, intermediate representations or policies from stronger teachers. Parameter-efficient adaptation changes a small fraction of weights. Quantization and pruning reduce deployment requirements. For an S*LM programme, most organizations should begin with adaptation and data engineering rather than aspire to pretrain a foundation model.

LoRA and QLoRA made adaptation accessible by training low-rank updates and, in QLoRA's case, backpropagating through a quantized base model. These methods lower hardware requirements and permit multiple role-specific adapters over one foundation. They do not remove the need for representative data, evaluation or lifecycle management. Adapter libraries can become operationally complex: versions interact with base-model updates, domain adapters can conflict with safety behavior, and a small weight file may encode consequential policy. [63, 62]

Distillation is especially relevant to agent roles. Traditional distillation transfers logits or representations. Step-by-step distillation uses rationales

or intermediate supervision. DeepSeek-R1 demonstrates that reasoning behavior can be transferred into smaller derivative models, while tool-use families show that action policies can be trained from generated trajectories. The S*LM opportunity is to distill decisions rather than prose: tool retrieval, function choice, argument construction, plan expansion, verifier repair and escalation. [67, 23, 29, 30]

Teacher data must be treated as evidence, not truth. A frontier model can produce diverse examples quickly, but it also propagates errors, stylistic habits and blind spots. Executable tasks offer an advantage because outputs can be run, type-checked or validated. APIGen’s verifiable data pipeline is important for this reason. In less formal domains, synthetic examples should be mixed with human cases, real failures, adversarial mutations and negative examples. The data-generation process should preserve why an example was accepted.

Curriculum matters more for small models because capacity is limited. A compact action model should first learn a stable output grammar, then common tools, then confusing pairs, irrelevant requests, multi-turn state and recovery. A formalizer should learn vocabulary and types before complex compositional mappings. A domain model should see canonical cases, boundary cases and explicit unknowns. The curriculum is not merely an optimization trick; it expresses the decomposition of competence.

Quantization changes the deployment regime. GPTQ, AWQ and SmoothQuant represent influential approaches to lower-precision inference. Quantization can make models practical on laptops, phones and edge devices, but its effect is not fully captured by average accuracy. Rare-token behavior, calibration, bias, structured output and tool selection may shift. The final deployed artifact, not the full-precision source model, must be evaluated on the actual role and hardware. [66, 64, 65, 68]

Pruning and sparse activation create another trade-off. Removing weights or activating only part of a model can reduce compute, while architectural changes may affect memory and serving complexity. MobileLLM explores designs optimized for sub-billion use. Granite and Nemotron families experiment with dense, hybrid and sparse variants.

Apple’s 2026 architecture combines specialization and hardware-aware optimization across on-device and cloud models. These examples are industrial signals rather than proof of one optimal architecture. They show that “parameter count” is increasingly an insufficient description of operational size. [3, 20, 21, 18]

Deployment cost depends on weights, context, prefill, decoding, batching, tool latency, network delay and retries. Long context can dominate memory through the key–value cache. A small model repeatedly fed an entire agent history may be slower and less reliable than a larger model receiving a compact, relevant state. External memory and context construction are therefore economic mechanisms as well as semantic ones.

Local deployment has four strategic benefits. It can reduce marginal inference cost at sufficient utilization. It can lower latency by avoiding network round trips. It can preserve operation when connectivity or cloud services are unavailable. It can keep sensitive data within a device or controlled infrastructure. These benefits must be balanced against engineering cost, hardware heterogeneity, update distribution, observability and the risk that underutilized local accelerators are more expensive than shared cloud capacity.

Runtimes such as llama.cpp, ONNX Runtime, ExecuTorch, LiteRT and MLX make local execution increasingly practical. vLLM focuses on efficient serving at larger shared scale. The runtime choice influences supported quantization, structured decoding, batching, device placement and monitoring. A model that is theoretically small but poorly supported on the target hardware may be an inferior choice to a slightly larger model with optimized kernels. [76, 77, 78, 79, 80, 75]

The economics of specialization include fixed costs. Teams must define the role, collect data, train or adapt the model, build validators, integrate it, monitor drift and maintain compatibility. The variable-cost savings matter only after enough volume. Specialization is attractive when a task is frequent, stable, measurable and expensive to serve with a general model. It is unattractive when the task changes faster than the specialized component can be maintained or when failures are too difficult to detect.

A useful break-even analysis therefore starts with workflow frequency and error cost, not model size. Suppose a document classifier runs millions of times and routes expensive downstream analysis. A compact specialist can justify substantial engineering because a small improvement in cost or routing quality compounds. Suppose a research agent encounters a rare theoretical problem once. Training a specialist is irrational; a general model and human review are appropriate. Repetition justifies specialization. Novelty justifies flexible escalation.

Routing introduces its own fixed and variable costs. Maintaining multiple models increases deployment complexity. The router can make mistakes. The model pool changes. Caching and batching become harder. LLMRouterBench’s finding that careful model curation can matter more than ever-larger ensembles is economically important. A small, well-chosen portfolio may outperform a crowded catalogue whose router cannot distinguish marginal differences. [48]

The two-timescale model offers a way to account for these costs. Slow acquisition invests in examples, adapters, procedures and tests. Fast execution uses the promoted capability repeatedly. The programme should report amortization: how many successful tasks are required before specialization repays its development and maintenance cost? It should also report the cost of failures and human review. A low inference bill can conceal an expensive operational process.

Industry signal. IBM explicitly argues that non-reasoning compact models can be preferable for enterprise instruction following and tool use because latency and stable token use matter. Google positions tiny models for task-specific customization. Mistral provides compact local variants under a permissive license. Apple invests in on-device and private-cloud specialization. NVIDIA argues strongly for small models as components in agentic systems and investigates small orchestrators. These claims align with vendor incentives, but the convergence is meaningful: efficient role allocation is becoming a product strategy. [20, 12, 13, 19, 18, 22, 34]

Academic signal. Research supports many enabling techniques but offers less certainty about system-level economics. Quantization, parameter-efficient fine-tuning, distillation and routing each show gains

under controlled conditions. Agent evaluation work warns that cost comparisons are misleading when baselines, retries and reproducibility differ. A serious S*LM study must therefore measure complete workflows on named hardware and include maintenance assumptions. [62, 64, 46, 49]

Research judgement. The objective is not the smallest model or the lowest token price. It is the lowest total cost for a task that succeeds, complies with policy, produces an auditable result and fails safely. Sometimes this will favor a local 270-million-parameter specialist. Sometimes it will favor a hosted frontier model. Frequently it will favor a cascade in which inexpensive components handle routine states and expensive components handle uncertainty.

Research hypothesis. A contract-based portfolio of compact specialists, symbolic tools and selective general-model escalation can lower total cost substantially in repetitive domains without reducing governed task success. The hypothesis should include all engineering and review costs. It fails if routing, maintenance and semantic errors erase the inference savings.

Conditional outcome. If supported, the economic advantage of S*LM will come from architectural learning. Organizations will gradually convert recurring expensive inference into reusable capabilities. They will reserve general models for the states where generality creates value. The resulting cost curve will resemble software automation: high initial formalization cost followed by low, predictable execution cost within a known envelope.

PART IV

Survey Synthesis and Research Programme

*The final part separates current industrial practice, academic evidence and the experiments required to turn S*LM from a useful pattern into a defensible research programme.*

CHAPTER 12

What Academia and Industry Are Actually Signalling

A survey becomes useful only when it distinguishes a durable signal from a crowded release calendar. The S*LM landscape contains hundreds of compact checkpoints, adapters, domain models, agent frameworks and neuro-symbolic proposals. Most are not independently important. The deeper pattern is a reorganization of where intelligence is placed in a system.

Industry's clearest signal is the rise of model families. Google's Gemma releases span general text models, efficient on-device variants, function calling, embeddings, safety, translation, medicine and interpretability. IBM's Granite family spans language, speech, vision, embeddings and guardians, with enterprise tool use and modular adapters as explicit themes. Mistral releases compact models in several sizes and modes for edge and local use. Apple trains distinct on-device and private-cloud models integrated with operating-system features. NVIDIA develops efficient model families and argues for compact orchestrators and workers. [15, 20, 19, 18, 21, 22]

This is not simply model proliferation. It is evidence that product builders no longer expect one checkpoint to be optimal for every stage. Embedding, moderation, translation, retrieval, function calling and open-ended synthesis have different latency, privacy, data and output requirements. The industry is rebuilding specialization inside the generative-model era rather than returning to a purely monolithic assistant.

A second industrial signal is on-device execution. Phones, laptops, vehicles and robots require low latency, offline availability and local data handling. MobileLLM, Phi, Llama 3.2, Gemma 3n, FunctionGemma, Ministral and Apple's foundation models occupy different points in this

space. Hardware-aware architecture, quantization and runtime integration are becoming model features rather than deployment afterthoughts. [3, 7, 11, 16, 14, 19, 17, 18]

A third industrial signal is the separation of action from conversation. FunctionGemma is explicitly an application component. Granite emphasizes tool calling. Apple's framework exposes constrained tool use. Commercial agent SDKs treat model calls, tools, state and human approval as separate primitives. The strongest production architectures are becoming less like chatbots with plugins and more like software runtimes with language interfaces.

Industry is also signalling caution, though less loudly than capability. Anthropic's guidance recommends workflows before agents and the simplest architecture that works. Its multi-agent research system is presented as expensive and appropriate for parallelizable research, not as a universal replacement. NIST and OWASP guidance emphasizes governance and security controls around models. These positions suggest that deployment experience is pushing against early assumptions that additional autonomy is always beneficial. [50, 51, 57, 58]

Academic research sends a complementary signal. The compact-model literature demonstrates that architecture, data quality, distillation and specialization can produce strong capabilities at modest scale. Yet papers also show uneven robustness and sensitivity to task design. The appropriate conclusion is not that scaling has stopped mattering. It is that parameter count is only one variable and often not the decisive one for a bounded role. [1, 2, 7, 6, 23]

Tool-use research shows that action competence can be trained. APIGen, xLAM, Hammer and ToolACE focus on executable data, irrelevant functions, multi-turn behavior and model-aware refinement. The deeper insight is that a tool caller learns a policy over an interface, not general intelligence. A narrow policy can be compact if the interface and training distribution are well defined. [29, 30, 31, 32, 33]

Neural-symbolic research shows that external computation can improve exactness where formalization is possible. Logic-LM, LINC and SatLM delegate deduction to solvers. LLM+P delegates search to a planner.

LLM-Modulo places external critics around generation. The durable insight is not that symbolic AI has defeated neural models. It is that semantic interpretation and exact computation have different strengths and should be separated when the boundary can be validated. [35, 36, 37, 38, 39]

Routing research sends a more skeptical signal. FrugalGPT and RouteLLM show that cascades can reduce cost and exploit complementarity. RouterBench and LLMRouterBench reveal that routing remains difficult, sophisticated methods often cluster near simple baselines and oracle performance remains distant. This means that “use a small model first and escalate” is an architecture, not yet a solved policy. [45, 46, 47, 48]

Multi-agent research is similarly mixed. Parallel and specialized systems can help when subtasks are separable and tools differ. MAST shows that coordination itself creates recurrent failure modes. Cost-controlled evaluation warns that complexity can appear as capability when baselines are weak. The field is gradually replacing the question “How many agents?” with “What independent capability or evidence does each agent contribute?” [52, 49, 51]

Memory research is moving from conversational metaphor to systems design. Surveys distinguish forms, functions and dynamics. Data-management studies examine representation, extraction, retrieval, routing and maintenance. This shift matters because long-horizon agents fail as much from stale, conflicting or mis-scoped state as from weak generation. [53, 54]

Reliability and security research offer the strongest corrective to capability enthusiasm. Average task success does not describe consistency, robustness or failure severity. Indirect prompt injection exploits the connection between untrusted language and tool authority. Privilege control and agentic security taxonomies move protection into the runtime. The convergence is clear: consequential autonomy requires controls that do not depend on the model recognizing every attack or its own uncertainty. [61, 59, 60, 58]

The survey also reveals what is not yet established. There is no generally reliable method for deciding when a compact model is sufficient.

There is no standard validation envelope that makes results comparable across model size, quantization, tool schemas and hardware. There is no broadly demonstrated method for faithful natural-language formalization outside restricted domains. There is no evidence that adding symbolic notation alone makes an agent trustworthy. There is no universal agent-memory architecture. There is no general proof that multi-agent systems are better than well-designed workflows.

These absences define the research opportunity. The field has many capable components and weak principles for composition. Models are evaluated separately from the runtimes that control them. Tool callers are evaluated separately from permissions. Formalizers are evaluated on syntax more often than semantic fidelity. Routers are evaluated on answer quality more often than governed task success. Memory is evaluated through downstream scores more often than update integrity. The S*LM programme should connect these evaluation layers.

Research judgement. Industry and academia are converging on modularity for different reasons. Industry is driven by product cost, hardware, latency, privacy and maintainability. Academia is driven by the limits of monolithic reasoning, the availability of formal tools and the need for interpretable experiments. Neither convergence proves that heterogeneous systems are always superior. It does justify treating the allocation of cognitive labour as a central research object.

The durable conclusion is narrower than “small is the future.” General models remain best for novel, underspecified work. Small models fit defined recurrent roles; specialized models fit roles that can be converted into data and tests; symbolic systems fit semantics that can be made explicit and checked. The runtime must allocate among them, preserve state and restrict authority.

Conditional outcome. Reliable allocation principles would make model ecosystems resemble computing ecosystems: general processors alongside specialized accelerators, compilers, databases and operating systems. Intelligence would reside in governed movement from ambiguous intent to justified action, not in one component.

CHAPTER 13

The S*LM Research Programme

The S*LM programme is not a proposal to build a smaller universal assistant. Its object is the heterogeneous agent runtime and the principles by which work is assigned to compact neural models, domain specialists, symbolic systems, deterministic software, larger models and humans. The programme succeeds only if it produces falsifiable knowledge about that allocation, not merely an impressive prototype.

The central research question is: **What is the least general component that can perform a given language-mediated role reliably within a declared validation and authority envelope?** This question connects model design, semantic parsing, tool use, planning, memory, security, human factors and economics. It also prevents the programme from assuming its conclusion. Some roles may resist specialization and continue to require broad models or human judgement.

The first research stream should construct a role taxonomy grounded in real workflows. Candidate roles include language identification, intent detection, entity and constraint extraction, ambiguity detection, translation, terminology normalization, summarization, claim extraction, sensitive-data detection, formalization, tool retrieval, function calling, planning, verification, memory writing, retrieval, explanation and escalation. The taxonomy should characterize semantic breadth, output entropy, consequence, verification strength, repetition, privacy and latency. The goal is to predict which substrate is appropriate, not to enumerate tasks indefinitely.

A useful experimental design begins with three domains that differ structurally. An enterprise operations domain provides recurrent intents, policies and databases. A scientific-review domain provides long documents, evidence, uncertainty and open-ended judgement. A software-engineering domain provides executable artifacts, tests and

evolving repositories. The same role taxonomy can then be tested against different semantics and verification possibilities.

The second stream should build **semantic intake contracts**. Experiments should compare a general-model baseline with layered systems using deterministic preprocessing, compact classification, structured extraction and explicit ambiguity states. Evaluation should follow the request through the entire workflow. The decisive metric is not intent classification in isolation, but whether goals, prohibitions, entities and conditions survive into the final action or report.

The third stream should study **invariant-preserving transformations**. Translation, summarization, redaction and normalization should be defined by domain-specific invariants. Test suites should mutate negation, quantity, modality, temporal order, entity identity, exceptions and uncertainty. Small models, adapters and general models should be compared under the same transformation profile. A result that reads better but changes a governing condition should be counted as a serious failure.

The fourth stream should investigate **verifier-aware formalization**. Compact models should translate language into schemas, controlled language, queries, logic, plans or code while receiving compiler and solver diagnostics. Evaluation must distinguish syntactic acceptance, execution success and semantic fidelity to the source. Ambiguous examples should permit multiple candidates or abstention. The programme should measure whether verifier-aware training improves repair without encouraging the model to optimize for superficial acceptance.

The fifth stream should develop **action models and typed tool ecosystems**. Real tool contracts should be transformed into training and adversarial data. The system should test tool retrieval, selection, argument construction, irrelevance, multi-turn state, recovery and policy compliance. FunctionGemma, xLAM, Hammer and ToolACE provide starting points, but the research target is not their reproduction. It is a general method for compiling an organization's tool surface into a validated compact action policy. [13, 29, 30, 31, 32]

The sixth stream should address **routing and escalation**. Current routing research shows complementarity and a persistent oracle gap. The

programme should combine task features with contract-derived signals: missing fields, novelty, verifier disagreement, failed repairs, sensitivity and consequence. It should compare learned routers with simple rules and calibrated cascades. The router should be evaluated on total governed task cost, including false escalation, silent failure and human review. [48, 61]

The seventh stream should build **agent-native memory** with data-system semantics. Memory objects should carry provenance, type, authority, sensitivity, version and expiry. Writes should be governed. Conflicts should remain visible. Retrieval should combine semantic relevance with authority and task dependency. Experiments should inject stale facts, poisoned summaries, contradictory preferences and lawful deletion. The research question is whether structured memory improves long-horizon reliability enough to justify its complexity. [53, 54]

The eighth stream should compare **workflow, single-agent and heterogeneous multi-agent architectures** under equal budgets and capabilities. Multi-agent designs should be admitted only when they introduce real parallelism, tool separation, privacy boundaries or independent verification. Communication should use typed artifacts where possible. Failure analysis should identify specification, coordination, memory and verification errors rather than report one aggregate score. [49, 52]

The ninth stream should establish **trust and authority envelopes**. Every component should declare what it can access and what actions its output may authorize. Threat models should cover prompt injection, malicious tool output, privilege escalation, data exfiltration, poisoned memory, invalid generated code and evaluator self-confirmation. Policy enforcement should remain outside the model. Human studies should measure calibration: whether users rely more appropriately on the system, not merely whether they prefer it. [57, 59, 60, 58]

The tenth stream should measure **economics and deployment**. Experiments should run on named cloud and local hardware, with actual quantized artifacts, context sizes and runtimes. Reports should include adaptation effort, serving utilization, tool latency, retry rate, maintenance and human review. A specialized model should be promoted only when its

total cost and reliability justify ownership. The project should explicitly document cases where a general hosted model remains the better engineering choice.

The research infrastructure should preserve two timescales. During acquisition, general models and coding agents may generate data, schemas, adapters, procedures and tests. Candidate capabilities enter a staging registry. Independent evaluation and domain review determine promotion. During execution, the runtime uses only named, versioned capabilities under policy. New behavior cannot silently become authoritative because a model improvised it once.

The core artifact should be a **capability package**. It contains the component identity, role contract, supported input distribution, output schema, training or source lineage, validation cases, known failures, resource profile, authority scope, escalation rules and version. For symbolic procedures it also contains assumptions and guarantee type. For model-based components it records the exact base model, adapter, quantization and decoding constraints. This package is more useful than a model card alone because it describes the model inside an operational role.

The programme requires a claim discipline. A result can be classified as demonstrated under a named workload, convergent across studies, plausible but unverified, or conditional on future evidence. A successful prototype in one company should not become a claim about enterprise AI generally. A benchmark improvement should not become a claim about trustworthy autonomy. A solver-backed result should state whether the uncertain step was formalization, data acquisition or both.

The main hypothesis is that recurrent language-mediated work contains enough stable structure to support progressive specialization without losing semantic fidelity. This hypothesis can fail in several ways. Task distributions may drift too quickly. The cost of data and validation may exceed inference savings. Small models may fail to recognize out-of-distribution cases. Formalization may introduce silent semantic loss. Routers may misallocate difficult states. Multiple components may create more cumulative error than a strong general model. These are not secondary risks; they are the falsification conditions.

A second hypothesis is that symbolic and deterministic components can substitute for some neural capacity by owning exact computation, search, state and policy. This also has clear limits. The symbolic representation may be incomplete. Real-world uncertainty may resist crisp predicates. Maintaining formal domain models may be expensive. Solver guarantees may apply to the wrong abstraction. The research contribution would be to identify where the substitution is advantageous, not to claim that symbolic methods absorb language understanding.

A third hypothesis is that specialization can improve trust by narrowing authority and making validation deeper. It fails when narrow models are deployed outside scope, when formal checks are mistaken for semantic truth, or when many opaque components create untraceable pipelines. Trust evaluation must therefore examine the complete trajectory and the human organization around it.

What success would mean. A successful programme would produce predictive design rules, reusable capability packages, reference runtimes, datasets derived from real contracts, failure taxonomies and evidence that system-level allocation improves cost, reliability or governance. It would demonstrate not one universal architecture but a method for choosing among architectures.

What success would not mean. It would not prove that large models are obsolete, that all language can be formalized, that symbolic systems are unbiased, or that a collection of small models constitutes general intelligence. Those claims are unnecessary. A substantial result would be showing that important classes of agent work can be made cheaper, more private, more inspectable and more reliable through disciplined division of labour.

The most promising startup opportunities lie where repetition, consequence and existing software meet. Enterprise workflows with defined tools and policies can support compact action models and governed execution. Scientific and legal document systems can support invariant-preserving extraction and source-linked formalization. Edge devices can support private local interpretation and action. Compliance and security can support explicit policy gates and provenance. The

defensible asset is not access to a generic foundation model. It is the accumulated contract data, validators, capability packages and integration knowledge for a domain.

The programme should remain open to a counterintuitive conclusion: sometimes the best S*LM component is ordinary software. A regular expression, database constraint, compiler, planner or reviewed function can outperform any language model for a stable operation. The star in S*LM should therefore be read as an invitation to composition, not as a demand that every component learn from text.

Closing position. The next important advance in agentic AI may not be a single model that internalizes every tool, policy and domain. It may be a science of deciding what should remain linguistic, what should be specialized, what should be formalized, what should be executed deterministically, what should be remembered and what should remain under human authority. S*LM names that science. Its ambition is not smaller intelligence. It is better-allocated intelligence.

APPENDIX A

A Role Taxonomy for S*LM

Components

This appendix turns the architectural argument into a practical classification. The role is the unit of evaluation. A single model may occupy several roles, but each role should retain its own contract, evidence and authority.

Role	Operational meaning and preferred validation
Language and modality identification	Detect the language, script, speech channel or visual input type before interpretation. Validate by language, code-switching pattern, noise condition and device environment.
Speech transcription	Convert audio into text while preserving speaker, timing, names, numbers and uncertainty. Evaluate on the actual acoustic and domain conditions rather than generic word-error rate alone.
Interface grounding	Map a linguistic reference to an element in a screen, image or document. Grounding accuracy must be separated from the success of the later action.
Coarse domain routing	Select the organizational or knowledge domain that owns a request. Hard data-governance rules should filter permissible destinations before learned routing.
Intent detection	Map a request to a supported operation or explicit unknown state. Test paraphrases, compound intents, prohibitions, conditions and out-of-scope cases.
Entity resolution	Connect names and descriptions to versioned internal objects. The result should preserve candidates and evidence when identity is ambiguous.
Constraint extraction	Identify temporal, quantitative, policy and exclusion constraints. Validate by source span and by mutation of negation, units and scope.
Ambiguity detection	Recognize when several interpretations remain plausible. The preferred output is a typed clarification need, not a confident guess.
Translation	Change language while preserving meaning, terminology, modality, names, quantities and structure. Domain-specific invariants are more important than general fluency.

Role	Operational meaning and preferred validation
Terminology normalization	Map variants to a controlled vocabulary without erasing legitimate distinctions. Every normalization should be reversible or linked to the source form.
Document segmentation	Identify sections, clauses, claims, tables, code and references. Deterministic layout and parser evidence can complement compact models.
Structured extraction	Populate a typed schema from text or images. Evaluate field correctness, omissions, source alignment and abstention.
Claim extraction	Represent claims, qualifiers, evidence and uncertainty rather than merely selecting salient sentences. Scientific and legal claims require domain profiles.
Summarization	Select and realize a shorter representation under explicit coverage and contradiction-preservation rules. Store source-linked structure as the authoritative artifact where possible.
Redaction and de-identification	Remove or transform sensitive information under a named privacy objective. Local execution helps, but re-identification and retention require separate controls.
Style and audience transformation	Change register, tone or reading level without strengthening claims. Compare the transformed text against protected semantic invariants.
Tool retrieval	Reduce a large catalogue to a policy-permitted candidate set. Measure recall of the correct tool, confusion among similar tools and behavior on irrelevant requests.
Function selection	Choose one or more tools for the current state. Validate at the level of completed policy-compliant state transitions, not isolated syntax.
Argument construction	Map entities and constraints into a tool schema. Type checking is necessary but semantic identity and units must be verified separately.
Plan proposal	Generate a coarse decomposition and identify dependencies or unknowns. Stable subplans should be delegated to workflows or symbolic planners.
Formal planning	Search action sequences under explicit preconditions and effects. The planner guarantees only what the domain and state representation express.
Database query construction	Translate intent into approved query objects or parameterized procedures. Direct unrestricted writes should remain outside model authority.
Logic or constraint formalization	Map language into predicates, rules or constraints. Evaluate source fidelity independently from solver acceptance.

Role	Operational meaning and preferred validation
Program synthesis	Generate executable transformations where narrower representations are insufficient. Require sandboxing, tests, resource limits and dependency controls.
Verifier and critic	Check a candidate against schemas, tests, evidence, policies or formal constraints. Declare exactly which property the verifier establishes.
Memory extraction	Select candidate information for persistence. The model proposes; a write policy assigns type, provenance, sensitivity, authority and expiry.
Memory retrieval and routing	Select state or evidence using relevance, authority, recency, dependency and permission. Similarity alone is insufficient.
Contradiction and staleness detection	Identify incompatible or superseded memories without forcing premature resolution. Preserve versions and evidence.
Explanation	Render actual evidence, formal artifacts and execution traces for a reader. Do not replace missing provenance with a plausible rationale.
Escalation	Package unresolved interpretation, evidence and failed checks for a stronger model, specialist or human. Evaluate both under- and over-escalation.
Safety and policy classification	Detect content or action categories under a named policy. The classifier supports, but does not replace, deterministic permissions and governance.
Human decision support	Organize evidence and trade-offs for a person with authority. Measure calibration and decision quality, not user preference alone.

APPENDIX B

Representative Model and Specialist Families

The entries below are an orientation map rather than a ranking. Sizes, licences and model cards change; the durable question is what role a family makes practical and which claims still require application-specific validation.

Model or family	S*LM relevance and caution
FunctionGemma, 270M	A clear example of a tiny specialized action model intended for customized function calling and local deployment. It is not positioned as a broad conversational assistant; application fine-tuning and validation are central.
Gemma 3, 270M to 4B compact range	Small general foundations for local adaptation, with a wider family that includes structured, multimodal and domain variants. Gemma terms are not the same as Apache or MIT licensing.
Gemma 3n, E2B and E4B	Multimodal models designed for efficient edge execution. Useful as local perception and language components; real device performance depends on modality, context and runtime.
Gemma 4 compact variants	Current-generation Gemma models include compact effective-size variants and function-calling support. Vendor comparisons should be treated as product evidence, not independent superiority claims.
EmbeddingGemma	A compact embedding specialist that illustrates how retrieval and memory can use a dedicated local component rather than a general generator.
TranslateGemma	A translation-focused family that supports the argument for separating language transformation from open-ended reasoning.
ShieldGemma	A safety-classification family. It can provide modular policy signals, but a classifier is not an access-control or authorization mechanism.
MedGemma	A medical text and multimodal family for developer applications. Domain specialization does not confer clinical authority and must be combined with evidence, validation and human governance.

Model or family	S*LM relevance and caution
Phi-3 and Phi-4 mini families	Compact Microsoft models emphasizing high-quality and synthetic data, reasoning and function-facing applications. Their practical value lies in strong local foundations, not in assuming all tasks fit a small context-independent model.
Qwen3 compact variants	Open-weight compact general and reasoning-capable models with broad ecosystem and tool-use support. The family is a useful base for routers and specialists; validation should distinguish thinking modes and deployment settings.
Llama 3.2, 1B and 3B	Widely supported compact models explicitly positioned for on-device use. Their ecosystem is a major advantage; licence terms and application-specific tool behavior require review.
SmolLM2 and SmolLM3	Hugging Face compact models with unusually open training information, valuable for research on small-model data, multilinguality, reasoning and reproducibility.
TinyLlama	An influential open small-model training effort and useful baseline for studying data and architecture at roughly one-billion-parameter scale.
MobileLLM	Research focused on optimizing sub-billion architectures for on-device tasks. It demonstrates that compact regimes benefit from architecture choices rather than simple shrinking.
IBM Granite 4.1, including 3B	Enterprise-oriented dense models emphasizing instruction following, tool calling and predictable latency, embedded in a broader family of speech, vision, embeddings and guardian models.
IBM Granite adapters and modular libraries	An industry signal toward treating AI capabilities as composable software functions. The value depends on compatibility, versioning and evaluation of each adapter.
Ministral 3, 3B, 8B and 14B	Apache-2.0 compact multimodal and multilingual models in base, instruct and reasoning variants, targeted at local and edge use. The 14B size is operationally small only for some deployment regimes.
DeepSeek-R1 distilled Qwen models	Evidence that reasoning-oriented behavior can be transferred into smaller derivatives. They are better viewed as compact reasoning workers than automatically safe action policies.
Apple on-device foundation models	Production examples of hardware-aware local language and multimodal intelligence with guided generation, tool calling and adapters. They are proprietary and tightly coupled to the Apple platform.
Apple AFM third generation	A family spanning on-device and private-cloud roles with architecture and quantization optimized by deployment target. Public evaluation is primarily vendor-reported.

Model or family	S*LM relevance and caution
NVIDIA Nemotron Nano variants	Efficient and sparse model families oriented toward agentic reasoning and deployment. Active parameters, total storage and serving stack should all be considered.
xLAM and xLAM-2 action models	Specialized API-selection and argument-generation families across several sizes. They show the importance of action data and multi-turn evaluation; checkpoint licences and exact use conditions vary.
Hammer	Compact function-calling models designed for on-device robustness, including mechanisms such as function masking and irrelevant-function training.
ToolACE and ToolACE-R	Data and refinement methods for function calling, especially relevant to improving small models through model-aware synthetic examples.
ToolLLaMA and ToolBench	A landmark tool-use system combining API data, retrieval and search. It is more important as an architectural and data precedent than as a current production recommendation.
Gorilla	An early API-connected model line that helped establish tool retrieval and grounded API generation as distinct research problems.
ToolOrchestra	A research direction in which a small orchestrator coordinates stronger models and tools. It supports the idea that compact models can allocate intelligence rather than reproduce it.
BioMistral	Open biomedical continued-pretraining models. Useful as domain foundations, but the broad medical label should be decomposed into specific tasks and risks.
LEGAL-BERT	A pre-generative example of domain specialization in legal language. It remains conceptually important because many legal tasks are classification and extraction rather than open generation.
FinGPT	An open financial-model initiative illustrating domain data and adaptation. Financial deployment requires temporal data controls, policy and evaluation beyond general model quality.

APPENDIX C

Symbolic, Formal, and Deterministic Backends

These systems are included because they can own computation that should not be simulated in prose. Their guarantees are always relative to the representation, assumptions and implementation supplied to them.

Backend or method	Agentic role and guarantee boundary
JSON Schema and typed records	Constrain fields, types and required structure. They establish representational validity, not the truth or appropriateness of values.
Grammar-constrained decoding	Restrict token generation to a grammar during decoding. XGrammar, Outlines and llama.cpp grammars improve syntax and efficiency but do not establish semantic safety.
Controlled natural language	Restrict language to improve deterministic parsing and review. Attempto Controlled English is a mature reference; domain coverage and author usability remain trade-offs.
Grammatical Framework	Separates abstract syntax from multilingual concrete syntax, useful for mapping several languages to shared domain representations when grammars are explicitly engineered.
SQL and parameterized database procedures	Execute data retrieval and updates under database semantics, transactions and access controls. Prefer approved query objects for consequential operations.
SPARQL and knowledge-graph queries	Retrieve explicit entities and relations. Graph completeness, ontology quality and provenance determine the legitimacy of conclusions.
Z3 and SMT solving	Check satisfiability and derive models under logical and theory constraints. The solver is exact relative to the encoded formula; natural-language formalization remains fallible.
clingo and answer-set programming	Support non-monotonic rules and combinatorial search. Appropriate for policies and discrete decisions when facts and rules can be stated explicitly.
PDDL and Fast Downward	Represent classical planning domains and search valid action sequences. The guarantee does not cover omitted preconditions, uncertain effects or an inaccurate initial state.
First-order theorem provers	Perform deductive reasoning over formal theories. LINC and related systems use language models as translators and provers as exact reasoners.

Backend or method	Agentic role and guarantee boundary
Probabilistic logic and DeepProbLog	Combine uncertain neural predicates with explicit logical structure. Useful where perception is uncertain but relations are formalizable.
Static analysis	Checks program properties without executing every path. It can validate generated code for defined classes of defects and should be paired with sandboxing and tests.
Unit, property and metamorphic testing	Provide executable evidence over selected examples and invariants. Passing tests does not prove correctness outside the test envelope.
Symbolic execution	Explores program paths under symbolic inputs. Valuable for generated procedures and policies, with scalability and environment-model limitations.
Model checking	Verifies temporal or state properties over a formal system. It is useful for bounded agent protocols when the transition model is explicit.
Constraint programming and optimization	Solve scheduling, allocation and configuration problems exactly or approximately under declared objectives. The objective function remains a human and organizational choice.
Rule and policy engines	Enforce organizational decisions deterministically. Progent illustrates a policy language around tool calls; generated policies still require validation.
Workflow and durable-execution engines	Own state transitions, retries, timeouts and compensation. They are often a better substrate than asking a model to remember procedural state in text.
Sandboxes and capability containers	Restrict filesystem, network, process and dependency effects of generated code or tools. They reduce blast radius rather than prove code harmless.
Cryptographic provenance and signed artifacts	Bind results to identities, versions and content hashes. They establish integrity and origin, not correctness of the content.
Human review and approval	Provides contextual judgement and authority that no formal component can assume. Its effectiveness depends on evidence presentation and reviewer calibration.

APPENDIX D

Deployment and Agentic Infrastructure

The technology stack determines whether a compact or specialized model is operationally useful. This appendix lists representative runtimes and orchestration systems without implying endorsement.

Technology	Role in an S*LM stack
llama.cpp	Local CPU, GPU and accelerator inference with broad quantization support and grammar-constrained generation.
vLLM	High-throughput shared serving, continuous batching and efficient attention memory management for open models.
ONNX Runtime	Cross-platform inference and hardware-provider abstraction for deploying compact models and classifiers.
ExecuTorch	PyTorch-oriented deployment runtime for mobile and embedded devices.
LiteRT	Google edge runtime for local inference on mobile and embedded targets.
MLX	Apple-silicon framework useful for local model adaptation and inference.
XGrammar	Efficient grammar-guided structured generation that can serve as an output-control layer.
Outlines	Structured-generation library spanning local and hosted model backends.
LangGraph	Graph and state-machine orchestration with checkpoints and human-intervention patterns.
Microsoft AutoGen	Framework for composing tool-using and multi-agent applications; complexity should be justified by real heterogeneity.
Hugging Face smolagents	Compact agent library aligned with open models and tool-based execution.
Pydantic AI	Typed model outputs, dependency injection and durable execution integrations for contract-oriented agents.
LlamaIndex workflows	Document, retrieval and workflow infrastructure for knowledge-intensive agents.

Technology	Role in an S*LM stack
CrewAI	Role-oriented multi-agent orchestration. Its value depends on whether roles correspond to distinct capabilities and evidence.
Google Agent Development Kit	Agent development and orchestration within Google's model and tool ecosystem.
OpenAI Agents SDK	Tool, handoff, tracing and guardrail abstractions for hosted-model agent systems.
Z3	SMT solver backend for constraints, policy checks, configuration and software reasoning.
clingo	Answer-set programming backend for rule-driven and combinatorial decisions.
Fast Downward	Classical planner for explicit PDDL domains.
Vector and relational databases	Persistent memory, provenance, transactions and access control. The database should own durable state rather than a prompt transcript.
Observability and tracing systems	Capture model, tool, state, policy, latency and cost events. Tracing becomes evidence only when identities and artifacts are versioned.

APPENDIX E

Research Hypotheses and Falsification Conditions

The programme is useful only if its claims can fail. This register separates plausible directions from established results and identifies evidence that would weaken each position.

Hypothesis	Evidence required and principal falsification condition
Minimum sufficient generality predicts model choice.	A role-characterization procedure should select portfolios with equal or better governed task success and lower total cost than size-first baselines across several domains. Failure to generalize beyond one workflow weakens the claim.
Repetition justifies progressive specialization.	Recurring trajectories should yield stable data, contracts and validators whose amortized value exceeds maintenance. Rapid drift or persistent semantic diversity would falsify broad applicability.
Compact action models can own routine tool use.	They must complete multi-step, policy-compliant state transitions with reliable abstention and recovery. High isolated function accuracy with poor end-to-end reliability is disconfirming.
Verifier-aware training improves formalization.	Models should preserve source meaning more often after training on compiler, solver and counterexample feedback. Merely increasing parser acceptance without semantic fidelity is failure.
External symbolic computation reduces required neural capacity.	Hybrid systems should match or exceed larger monolithic models on formalizable tasks at lower cost while exposing guarantees. If translation errors dominate, the substitution advantage is limited.
Contract-first runtimes improve trust and maintainability.	With the same base models, contract systems should localize failures, contain privilege and permit component replacement better than prompt-centric baselines.
Agent-native memory improves long-horizon reliability.	Provenance, versioning, conflict and expiry should reduce stale-state and contamination failures. If complexity adds more errors than it prevents, the hypothesis fails.
Routing can safely concentrate general-model use on novel states.	Routers must reduce cost without increasing severe silent failures and should outperform simple rules under distribution shift. Persistent oracle gaps constrain the claim.

Hypothesis	Evidence required and principal falsification condition
Heterogeneous multi-agent systems help when warrants differ.	Gains should correlate with real parallelism, independent tools or verification, not fictional roles. Equal-budget comparisons against simpler workflows are mandatory.
Local compact models create strategic privacy value.	Deployments should reduce external data exposure while maintaining access control, retention and update governance. Locality without data governance is not supporting evidence.
Specialization can improve fairness and alignment testability.	Narrow roles should permit deeper subgroup and policy evaluation. New bias from domain data, distillation or quantization must be measured; improvement is not assumed.
Capability packages become durable organizational assets.	Versioned contracts, data, validators and provenance should survive model replacement and improve future development. If knowledge remains trapped in prompts and tacit integration, the programme has not succeeded.

References

The bibliography prioritizes primary papers, official model documentation and maintained technical projects. Vendor pages document released capabilities and positions; they are not treated as independent comparative evidence.

- [1] Z. Lu et al., “Small Language Models (SLMs) Can Still Pack a Punch: A Survey,” arXiv:2501.05465, revised 2026. <https://arxiv.org/abs/2501.05465>
- [2] Y. Liu et al., “What Is the Role of Small Models in the LLM Era: A Survey,” arXiv:2409.06857, 2024–2025. <https://arxiv.org/abs/2409.06857>
- [3] Z. Liu et al., “MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases,” arXiv:2402.14905, 2024. <https://arxiv.org/abs/2402.14905>
- [4] P. Zhang et al., “TinyLlama: An Open-Source Small Language Model,” arXiv:2401.02385, 2024. <https://arxiv.org/abs/2401.02385>
- [5] Hugging Face, “SmolLM: Blazingly Fast and Remarkably Powerful,” 2024. <https://huggingface.co/blog/smolLM>
- [6] Hugging Face, “SmolLM3: Smol, Multilingual, Long-Context Reasoner,” 2025. <https://huggingface.co/blog/smolLM3>
- [7] M. Abdin et al., “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone,” arXiv:2404.14219, 2024. <https://arxiv.org/abs/2404.14219>
- [8] Microsoft, “Empowering Innovation: The Next Generation of the Phi Family,” 2025. <https://azure.microsoft.com/en-us/blog/empowering-innovation-the-next-generation-of-the-phi-family/>
- [9] Qwen Team, “Qwen3 Technical Report,” arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>
- [10] Qwen Team, “Qwen3: Think Deeper, Act Faster,” 2025. <https://qwenlm.github.io/blog/qwen3/>
- [11] Meta AI, “Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models,” 2024. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
- [12] Google, “Introducing Gemma 3 270M: The Compact Model for Hyper-Efficient AI,” 2025. <https://developers.googleblog.com/en/introducing-gemma-3-270m/>
- [13] Google AI for Developers, “FunctionGemma Model Overview,” 2025–2026. <https://ai.google.dev/gemma/docs/functiongemma>
- [14] Google, “On-Device Function Calling in Google AI Edge Gallery,” 2026. <https://developers.googleblog.com/on-device-function-calling-in-google-ai-edge-gallery/>
- [15] Google AI for Developers, “Gemma Releases,” current to August 2026. <https://ai.google.dev/gemma/docs/releases>
- [16] Google AI for Developers, “Gemma 3n Model Overview,” 2025. <https://ai.google.dev/gemma/docs/gemma-3n>
- [17] Apple Machine Learning Research, “Apple Intelligence Foundation Language Models Tech Report 2025,” 2025. <https://machinelearning.apple.com/research/apple-foundation-models-tech-report-2025>
- [18] Apple Machine Learning Research, “Introducing the Third Generation of Apple Foundation Models,” 2026. <https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models>
- [19] Mistral AI, “Introducing Mistral 3,” 2025. <https://mistral.ai/news/mistral-3/>

- [20] IBM Research, “Introducing the IBM Granite 4.1 Family of Models,” 2026.
<https://research.ibm.com/blog/granite-4-1-ai-foundation-models>
- [21] NVIDIA, “Inside NVIDIA Nemotron 3,” 2025–2026.
<https://developer.nvidia.com/blog/inside-nvidia-nemotron-3-techniques-tools-and-data-that-make-it-efficient-and-accurate/>
- [22] A. Belousov et al., “Small Language Models are the Future of Agentic AI,” arXiv:2506.02153, 2025.
<https://arxiv.org/abs/2506.02153>
- [23] DeepSeek-AI, “DeepSeek-R1,” arXiv:2501.12948 and official repository, 2025.
<https://github.com/deepseek-ai/DeepSeek-R1>
- [24] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” arXiv:2302.04761, 2023. <https://arxiv.org/abs/2302.04761>
- [25] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2022–2023. <https://arxiv.org/abs/2210.03629>
- [26] E. Karpas et al., “MRKL Systems: A Modular, Neuro-Symbolic Architecture That Combines Large Language Models, External Knowledge Sources and Discrete Reasoning,” arXiv:2205.00445, 2022.
<https://arxiv.org/abs/2205.00445>
- [27] Y. Qin et al., “ToolLLM: Facilitating Large Language Models to Master 16,000+ Real-World APIs,” arXiv:2307.16789, 2023. <https://arxiv.org/abs/2307.16789>
- [28] S. Patil et al., “Gorilla: Large Language Model Connected with Massive APIs,” arXiv:2305.15334, 2023. <https://arxiv.org/abs/2305.15334>
- [29] Z. Liu et al., “APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets,” arXiv:2406.18518, 2024. <https://arxiv.org/abs/2406.18518>
- [30] J. Zhang et al., “xLAM: A Family of Large Action Models to Empower AI Agent Systems,” arXiv:2409.03215, 2024–2025. <https://arxiv.org/abs/2409.03215>
- [31] Q. Lin et al., “Hammer: Robust Function-Calling for On-Device Language Models via Function Masking,” arXiv:2410.04587, 2024. <https://arxiv.org/abs/2410.04587>
- [32] X. Liu et al., “ToolACE: Winning the Points of LLM Function Calling,” arXiv:2409.00920, 2024.
<https://arxiv.org/abs/2409.00920>
- [33] X. Liu et al., “ToolACE-R: Model-Aware Iterative Training and Adaptive Self-Refinement for Tool Learning,” arXiv:2504.01400, 2025–2026. <https://arxiv.org/abs/2504.01400>
- [34] H. Su et al., “ToolOrchestra: Elevating Intelligence via Efficient Model and Tool Orchestration,” arXiv:2511.21689, 2025. <https://arxiv.org/abs/2511.21689>
- [35] L. Pan et al., “Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning,” arXiv:2305.12295, 2023. <https://arxiv.org/abs/2305.12295>
- [36] S. Olausson et al., “LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers,” arXiv:2310.15164, 2023. <https://arxiv.org/abs/2310.15164>
- [37] Y. Ye et al., “SatLM: Satisfiability-Aided Language Models Using Declarative Prompting,” arXiv:2305.09656, 2023. <https://arxiv.org/abs/2305.09656>
- [38] B. Liu et al., “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency,” arXiv:2304.11477, 2023. <https://arxiv.org/abs/2304.11477>
- [39] S. Kambhampati et al., “LLM-Modulo: Strengthening LLMs through External Verifiers,” arXiv:2402.01817, 2024. <https://arxiv.org/abs/2402.01817>
- [40] K. Valmeekam et al., “On the Planning Abilities of Large Language Models: A Critical Investigation,” arXiv:2305.15771 and PlanBench, 2023–2024. <https://arxiv.org/abs/2305.15771>
- [41] R. Manhaeve et al., “DeepProbLog: Neural Probabilistic Logic Programming,” NeurIPS 2018.
<https://arxiv.org/abs/1805.10872>
- [42] S. Dong et al., “XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models,” arXiv:2411.15100, 2024–2025. <https://arxiv.org/abs/2411.15100>

- [43] B. Geng et al., “Generating Structured Outputs from Language Models,” arXiv:2501.10868, 2025.
<https://arxiv.org/abs/2501.10868>
- [44] Outlines Documentation, structured generation across local and hosted models.
<https://dottxt-ai.github.io/outlines/latest/>
- [45] L. Chen et al., “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” arXiv:2305.05176, 2023. <https://arxiv.org/abs/2305.05176>
- [46] I. Ong et al., “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv:2406.18665, 2024.
<https://arxiv.org/abs/2406.18665>
- [47] L. Hu et al., “RouterBench: A Benchmark for Multi-LLM Routing System,” arXiv:2403.12031, 2024.
<https://arxiv.org/abs/2403.12031>
- [48] H. Li et al., “LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing,” arXiv:2601.07206, 2026. <https://arxiv.org/abs/2601.07206>
- [49] S. Kapoor et al., “AI Agents That Matter,” arXiv:2407.01502, 2024. <https://arxiv.org/abs/2407.01502>
- [50] Anthropic, “Building Effective AI Agents,” 2024.
<https://www.anthropic.com/research/building-effective-agents>
- [51] Anthropic, “How We Built Our Multi-Agent Research System,” 2025.
<https://www.anthropic.com/engineering/multi-agent-research-system>
- [52] M. Cemri et al., “Why Do Multi-Agent LLM Systems Fail?,” arXiv:2503.13657, 2025.
<https://arxiv.org/abs/2503.13657>
- [53] “Memory in the Age of AI Agents,” arXiv:2512.13564, 2025–2026. <https://arxiv.org/abs/2512.13564>
- [54] W. Zhou et al., “Are We Ready for an Agent-Native Memory System?,” arXiv:2606.24775, 2026.
<https://arxiv.org/abs/2606.24775>
- [55] M. Sharma et al., “Towards Understanding Sycophancy in Language Models,” arXiv:2310.13548, revised 2025. <https://arxiv.org/abs/2310.13548>
- [56] J. Batzner et al., “Sycophancy Claims about Language Models: The Missing Human-in-the-Loop,” arXiv:2512.00656, 2025. <https://arxiv.org/abs/2512.00656>
- [57] NIST, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,” NIST AI 600-1, 2024.
<https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- [58] OWASP GenAI Security Project, “Top 10 for Agentic Applications for 2026.”
<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- [59] Q. Zhan et al., “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” arXiv:2403.02691, 2024. <https://arxiv.org/abs/2403.02691>
- [60] T. Shi et al., “Progent: Programmable Privilege Control for LLM Agents,” arXiv:2504.11703, 2025.
<https://arxiv.org/abs/2504.11703>
- [61] S. Rabanser et al., “Towards a Science of AI Agent Reliability,” arXiv:2602.16666, 2026.
<https://arxiv.org/abs/2602.16666>
- [62] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” arXiv:2305.14314, 2023.
<https://arxiv.org/abs/2305.14314>
- [63] E. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685, 2021.
<https://arxiv.org/abs/2106.09685>
- [64] J. Lin et al., “AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration,” arXiv:2306.00978, 2023–2024. <https://arxiv.org/abs/2306.00978>
- [65] G. Xiao et al., “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” arXiv:2211.10438, 2022–2023. <https://arxiv.org/abs/2211.10438>
- [66] E. Frantar et al., “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” arXiv:2210.17323, ICLR 2023. <https://arxiv.org/abs/2210.17323>

- [67] C. Hsieh et al., “Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes,” arXiv:2305.02301, 2023. <https://arxiv.org/abs/2305.02301>
- [68] E. Kirsten et al., “The Impact of Inference Acceleration on Bias of LLMs,” arXiv:2410.22118, revised 2025. <https://arxiv.org/abs/2410.22118>
- [69] H. Yang et al., “FinGPT: Open-Source Financial Large Language Models,” arXiv:2306.06031, 2023. <https://arxiv.org/abs/2306.06031>
- [70] I. Chalkidis et al., “LEGAL-BERT: The Muppets Straight Out of Law School,” arXiv:2010.02559, 2020. <https://arxiv.org/abs/2010.02559>
- [71] A. Labrak et al., “BioMistral: A Collection of Open-Source Pretrained Large Language Models for Medical Domains,” arXiv:2402.10373, 2024. <https://arxiv.org/abs/2402.10373>
- [72] Google Health AI Developer Foundations, “MedGemma,” 2025–2026. <https://developers.google.com/health-ai-developer-foundations/medgemma>
- [73] S. Siegel et al., “CORE-Bench: Fostering the Credibility of Published Research through a Computational Reproducibility Agent Benchmark,” arXiv:2409.11363, revised 2026. <https://arxiv.org/abs/2409.11363>
- [74] Anthropic, “Paving the Way for AI Agents in Biology,” 2026. <https://www.anthropic.com/research/agents-in-biology>
- [75] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP 2023; vLLM project. <https://github.com/vllm-project/vllm>
- [76] ggml-org, “llama.cpp,” open-source local inference runtime. <https://github.com/ggml-org/llama.cpp>
- [77] Microsoft, “ONNX Runtime,” cross-platform inference runtime. <https://onnxruntime.ai/>
- [78] PyTorch, “ExecuTorch,” on-device AI runtime. <https://pytorch.org/executorch/>
- [79] Google, “LiteRT and Google AI Edge,” on-device inference. <https://ai.google.dev/edge/litert>
- [80] Apple, “MLX,” array framework for Apple silicon. <https://github.com/ml-explore/mlx>
- [81] Microsoft Research, “Z3 Theorem Prover.” <https://github.com/Z3Prover/z3>
- [82] Potassco, “clingo: A Grounder and Solver for Logic Programs.” <https://potassco.org/clingo/>
- [83] Fast Downward Planning System. <https://www.fast-downward.org/>
- [84] Attempto Controlled English and Attempto Parsing Engine. <https://attempto.ifi.uzh.ch/site/>
- [85] Grammatical Framework. <https://www.grammaticalframework.org/>
- [86] P. Liang, “Learning Executable Semantic Parsers for Natural Language Understanding,” Communications of the ACM, 2016. <https://dl.acm.org/doi/10.1145/2866568>